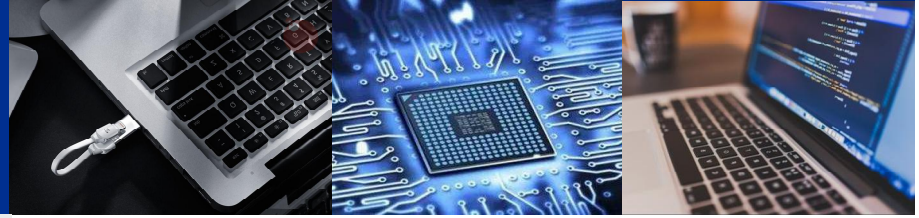


3 寻址方式与指令系统

- 指令系统概述
- 8086寻址方式
- 8086指令系统
- 例题解析



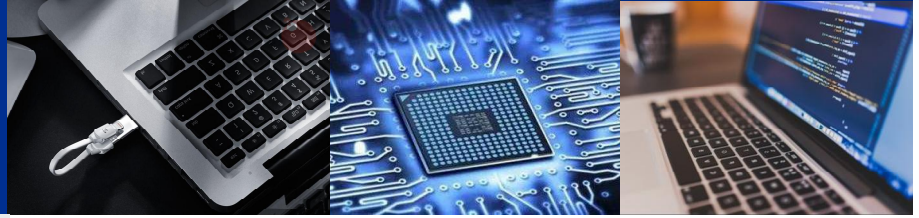
3.1指令系统概述

3.1.1指令的基本概念

- **指令**是要求计算机执行特定操作的命令。
- 计算机所能执行的全部指令的集合称为**指令系统**。
- 指令以二进制编码的形式存放在存储器中，用二进制编码形式表示的指令称为**机器指令**。
- 用符号表示的指令称为**符号指令**，汇编语言程序中的指令就是符号指令。具有直观、易理解和可帮助记忆的特点。

表4.1 SUB指令

机器指令（二进制）	十六进制	汇编指令	功能
01001011	4BH	DEC BX	将寄存器BX的内容减1



3.1指令系统概述

3.1.2 指令格式

汇编指令由**操作码**字段和**操作数**字段构成。

操作码字段	操作数字段

指令字长度：一个指令字中包含二进制代码的位数。

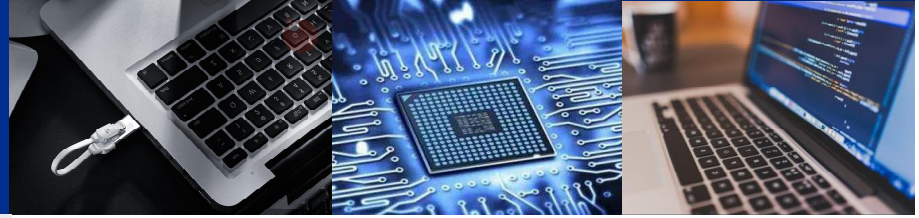
机器字长：指计算机能直接处理的二进制数据的位数，它决定了计算机的运算精度。

单字长指令：指令字长度等于机器字长的指令。

半字长指令：指令字长度等于半个机器字长度的指令。

双字长指令：指令字长度等于2个机器字长度的指令。

在一个指令系统中，如果各种指令字长度是相等的，称为等长指令字结构；如果各种指令字长度随指令功能而异，比如有的指令是单字长指令，有的指令是双字长指令，就称为变字长指令字结构。



3.1指令系统概述

3.1.3 8086汇编语言格式

8086 指令的一般格式:

标号: 操作码 操作数 ; 注释

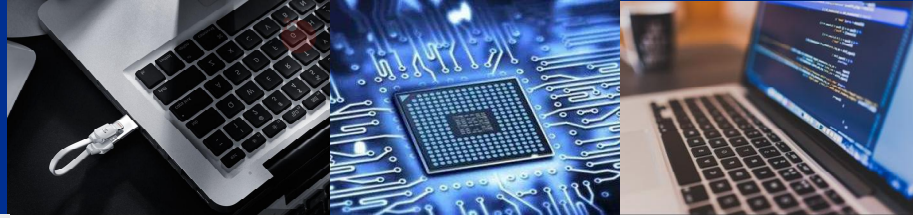
其中，标号和注释都是可选项。标号表示该指令在代码段中的偏移地址; 注释对该指令进行说明，不参加指令的执行。

- **操作码**：说明指令的性质与**功能**。
- **操作数**：参与运算的**操作数**或存放操作数的**地址**。（数值、寄存器、存储器）
- 分固定长度和可变长度。X86系列CPU（CISC）采用可变长度，ARM（RISC）采用固定长度。
- 8086CPU的机器指令长度为1～6个字节。

如：汇编指令MOV AX，05C7H的机器指令编码为B8C705H，转换成二进制为：

1011 1000 1100 0111 0000 0101B

3 寻址方式与指令系统



3.1指令系统概述

1. 双操作数指令（两地址指令）

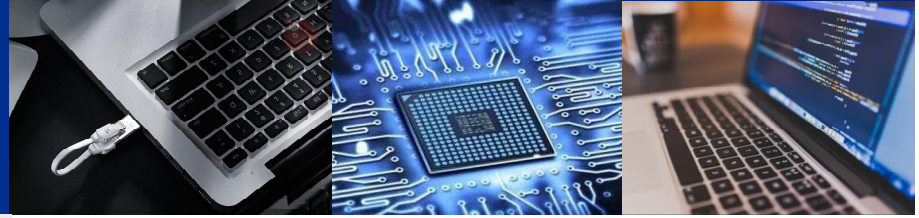
```
MOV   AX，5      ；传送指令。
ADD   AX，BX     ；加法指令。
```

目的操作数　源操作数

2. 单操作数指令（一地址指令）

```
INC  AX          ；加1指令。
MUL              ；乘法指令。
PUSH  AX         ；进栈指令。
JMP  LA1         ；无条件转移指令。
```

目的操作数



3.1指令系统概述

3. 无操作数指令（零地址指令）

CBW ； 字节转换为字指令

CLC ； 进位标志CF清零

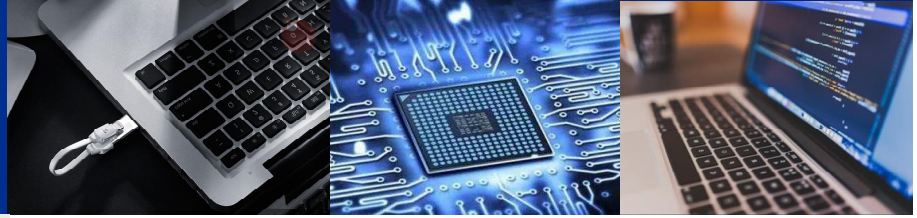
NOP ； 不操作指令

HLT ； 停机指令

4. 三操作数指令（三地址指令）

IMUL EBX，[ESI]，7 ； 乘法指令。

（80386机器指令）



3.2 8086寻址方式

操作数存在方式

- **操作数包含在指令中**——即指令的操作数部分就包含着操作数本身。

```
MOV  AX, 1234
ADD  AL, 2
```

- **操作数包含在CPU的某一个内部寄存器中** —— 这时指令中的操作数是CPU内部的某一个寄存器

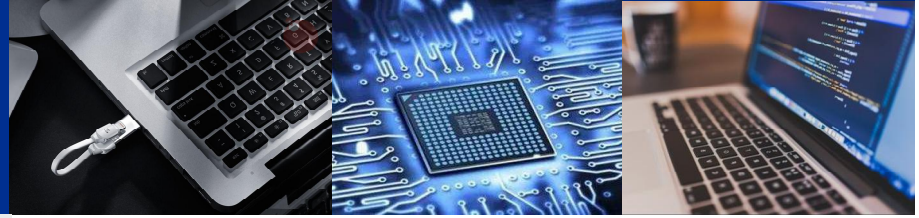
```
MOV  DS, AX
```

- **操作数在内存的数据区中**—— 这时指令中的操作数包含着此操作数的地址

```
MOV  AX,[2000]
MOV  buffer[SI],AX
```

- **操作数在I/O端口寄存器中。** ——这时指令中的操作数包含着此操作数的所在端口地址

```
IN AL,n        ;n是端口号
```



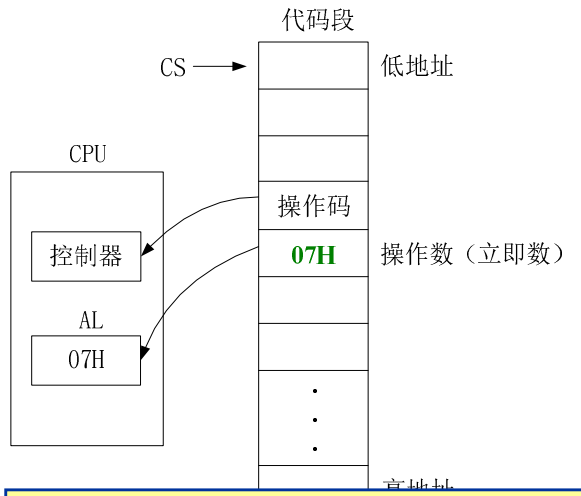
3.2 8086寻址方式

1. 立即寻址（Immediate Addressing）

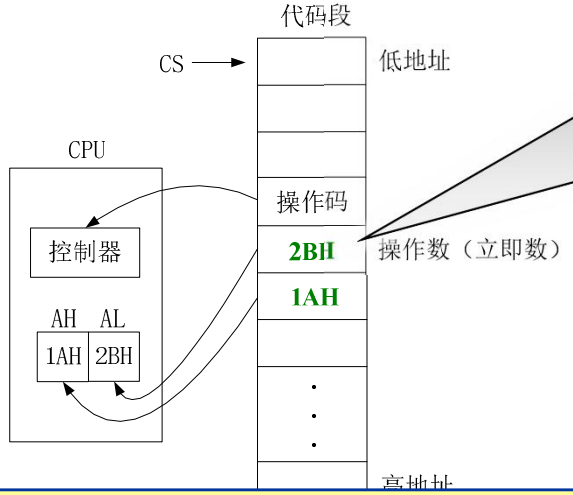
操作数直接存放在指令中，紧跟在操作码之后，作为指令的一部分，存放在代码段里，这种操作数称为**立即数**。常用来给寄存器赋值。

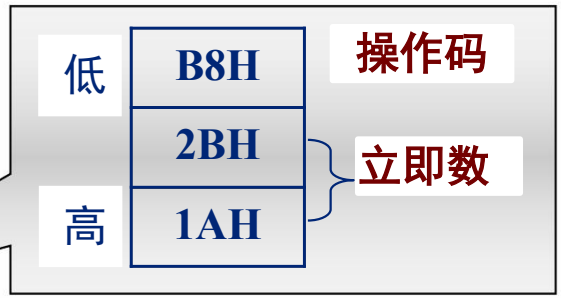
指令：MOV AL，07H

MOV AL, ‘A’

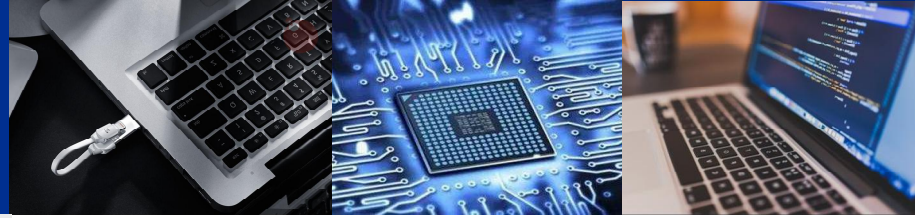


指令：MOV AX，1A2BH





- 立即数只能做源操作数。 （×）MOV 3，AL
- 源、目的操作数的字长必须一致。 （×）MOV AH, 3064H



3.2 8086寻址方式

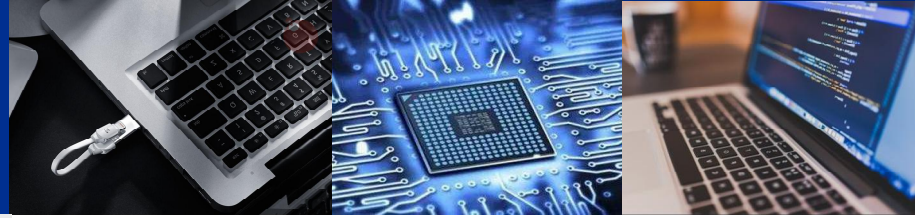
2. 寄存器寻址：（Register Addressing）

- 操作数存放在CPU的内部寄存器reg中：
  - 8位寄存器**R8**：

    AH、AL、BH、BL、CH、CL、DH、DL
  - 16位寄存器**R16**：

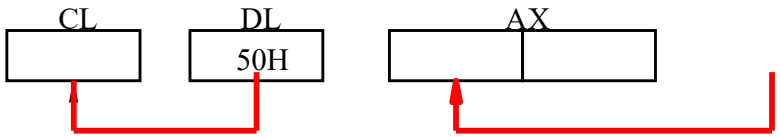
    AX、BX、CX、DX、SI、DI、BP、SP
  - 4个段寄存器**Seg**：

    CS、DS、SS、ES
- 寄存器名表示其内容（操作数）



3.2 8086寻址方式

【例】 MOV CL，DL； （DL）=50H
MOV AX，BX； （BX=1234H）目的、源操作数采用寄存器寻址



说明：

▲ **由于操作数在寄存器中**，指令执行时，操作就在CPU 的内部进行，不需要访问存储器来取得操作数，因而**执行速度快**。

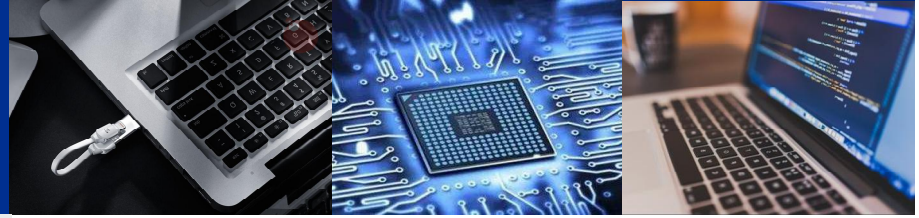
▲ 寄存器符号比内存地址短，汇编后**机器码长度最短**。

▲ 寄存器寻址方式既可用于DST，也可用于SRC，还可以两者都用寄存器寻址。

★ 当指令中的 DST和 SRC均为寄存器时， 必须采用同样长度的寄存器；

★ 两个操作数不能同时为段寄存器；

★ 目的操作数不能是代码段寄存器。



3.2 8086寻址方式

3. 存储器寻址（Memory Addressing）

◆ 操作数存放在存储器中，CPU取出指令后，为了获得操作数（对于源操作数）或操作数的存放地址（对于目的操作数）还要再次访问存储器。

◆ 指令中以**逻辑地址（有效地址EA）**表示操作数存放的位置，可能存放在存储器的任意一个逻辑段中，CPU必须计算出操作数的**物理地址**才能完成存储单元的读、写。

◆ 存储器寻址既可用于源操作数，也可用于目的操作数，但**两者不能同时使用**。

◆ 用方括号对 **[]** 表示存储器寻址。

■ **逻辑地址LA**：段地址:偏移地址

■ **物理地址PA**：段地址×16 + 偏移地址

指令中，不允许使用AX、CX、DX等作偏移地址

$$EA = \left\{\begin{matrix}BX\\BP\end{matrix}\right\} + \left\{\begin{matrix}SI\\DI\end{matrix}\right\} + DISP$$

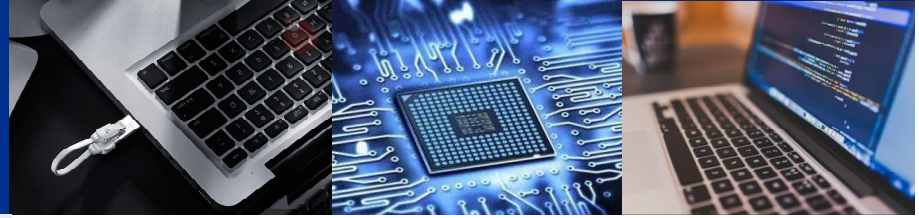
存放在指令中的一个8位、16或32位的数，但它不是立即数，而是一个地址

存储器操作数具有三个属性：

■ **段 地 址**：操作数所在的逻辑段的段地址。

■ **偏移地址**：相对段地址的偏移量。

■ **数据类型**：操作数是一个字节/字，用BYTE PTR/WORD PTR区分。



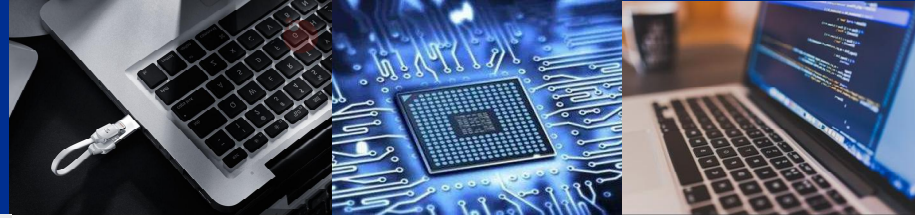
3.2 8086寻址方式

基址　变址　偏移量

$$EA = \begin{pmatrix} BX \\ BP \end{pmatrix} + \begin{pmatrix} SI \\ DI \end{pmatrix} + DISP$$

①　②　③

寻址方式	有效地址EA	默认的段寄存器
直接寻址（③）	[DISP]	DS
寄存器间接寻址（①/②）	[BX]	DS
	[BP]	SS
	[SI]	DS
	[DI]	DS
寄存器相对寻址（①+③）	[BX + DISP]	DS
	[BP + DISP]	SS
基址加变址寻址（①+②）	[BX+SI/DI]	DS
	[BP+SI/DI]	SS
相对基址加变址寻址（①+②+③）	[BX + SI + DISP（或无DISP）]	DS
	[BX + DI + DISP（或无DISP）]	DS
	[BP + SI + DISP（或无DISP）]	SS
	[BP + DI + DISP（或无DISP）]	SS



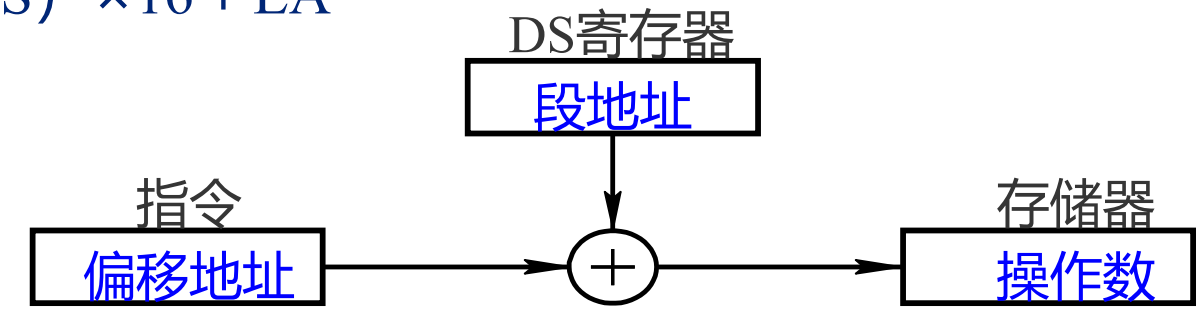
3.2 8086寻址方式

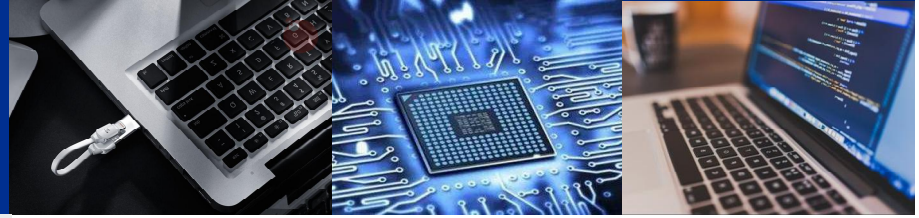
①直接寻址

有效地址EA在指令中**直接**给出，默认段是**DS**。

EA：操作数的偏移地址

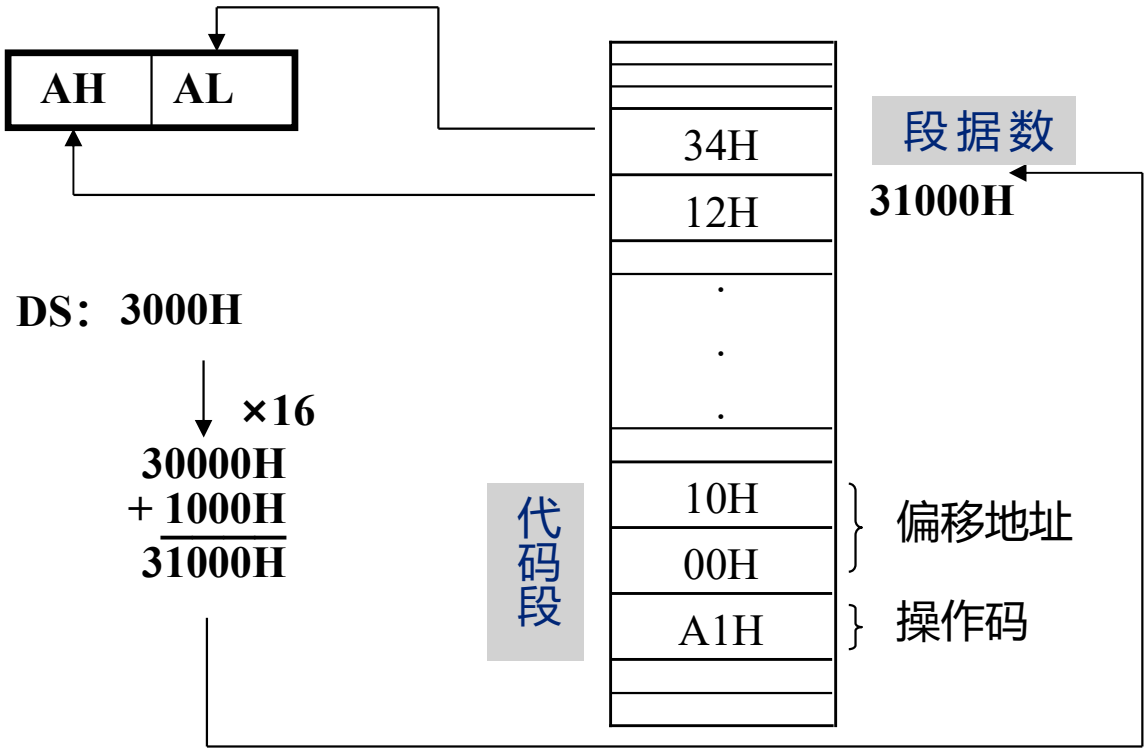
PA：（DS）×16 + EA





3.2 8086寻址方式

指令MOV AX，[1000H]的示意图

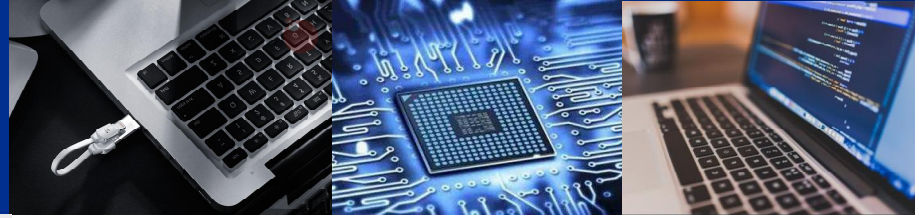


执行结果
(AX)=1234H

- ◆根据指令中给出的有效地址，得到存储单元的物理地址：

  PA = DS×16 + 1000H = 31000H

- ◆把该内存单元中的内容送到AX中；
- ◆字在内存中占两个内存单元，低字节在前（**低地址**），高字节在后（**高地址**）；并以低字节的地址作为字的地址。



3.2 8086寻址方式

★ 直接寻址允许数据存于**附加段、堆栈段、代码段**，这称为“**段跨越**”。

但需要使用段跨越前缀，例如 MOV AX, ES: [2000H]中，**数据存于附加段**中，

操作数物理地址PA为：ES x 16 + 0100H

★ 在汇编语言指令中，可以用**符号地址代替数值地址**

【例】 **VALUE DB 12H，34H，56H**；数据定义

MOV AL，VALUE

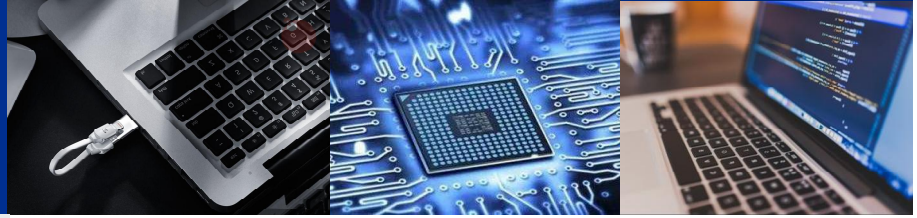
MOV AL，[VALUE]

但要注意变量的属性

MOV AH, VALUE ；**与 MOV AH,[VALUE] 等效**

MOV AX, VALUE ×

MOV AX, WORD PTR VALUE √



3.2 8086寻址方式

②寄存器间接寻址

▲ 把内存操作数的有效地址存储于寄存器中，指令中给出存放地址的寄存器名；

▲ 因为**有效地址是16位**，存放地址所使用的**寄存器必须是16位的**：

▲ 可以用于间接寻址的寄存器有**基址寄存器**BX、BP 和**变址寄存器** SI、DI。

▲ 为了区别于寄存器寻址，寄存器名用“ [ ] ”括起。

☆ **不同的寄存器所隐含对应的段不同。**

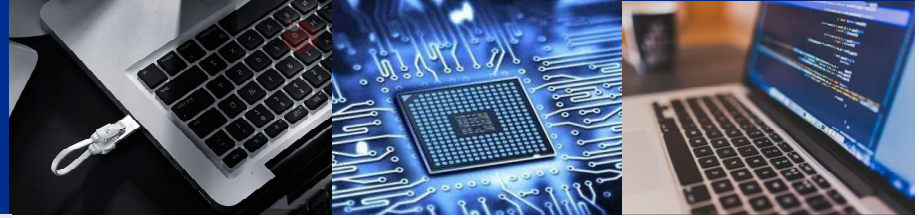
采用**SI、DI、BX** 寄存器，数据存于**数据段**中；

采用 **BP** 寄存器，数据存于**堆栈段**中；

操作数的物理地址计算式为：

PA = DS X16 + SI / DI / BX

PA = SS x 10H + BP



3.2 8086寻址方式

有效地址EA由寄存器（BX，BP，SI，DI）间接给出。

当EA = （BX）/（SI）/（DI）时：PA = （DS）×16 + EA

当EA = （BP）时：PA = （SS）×16 + EA

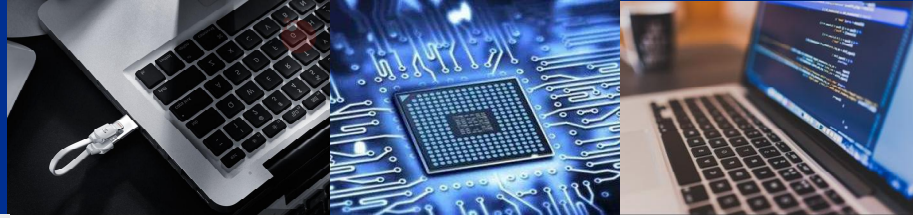
```
MOV  AX, [BX]                ; 源操作数在数据段，EA = BX，
PA = DS×16 + EA
MOV  AX, [DI]                ; 源操作数在数据段，EA = DI，
PA = DS×16 + EA
```

不允许使用AX、CX、DX、SP存放EA：**（×）MOV AX, [CX]**

源、目的操作数不能同时带方括号：**（×）MOV [BX], [SI]**

立即数和存储器寻址方式同时使用时应明示数据类型：

（×）MOV [BX]，20H



3.2 8086寻址方式

③ 寄存器相对寻址

☆ 寄存器相对寻址时，操作数的有效地址分为两部分：

▲ 一部分存于**寄存器**中，指令中给出该寄存器名；

▲ 另一部分以**偏移量**的方式直接在指令中给出。

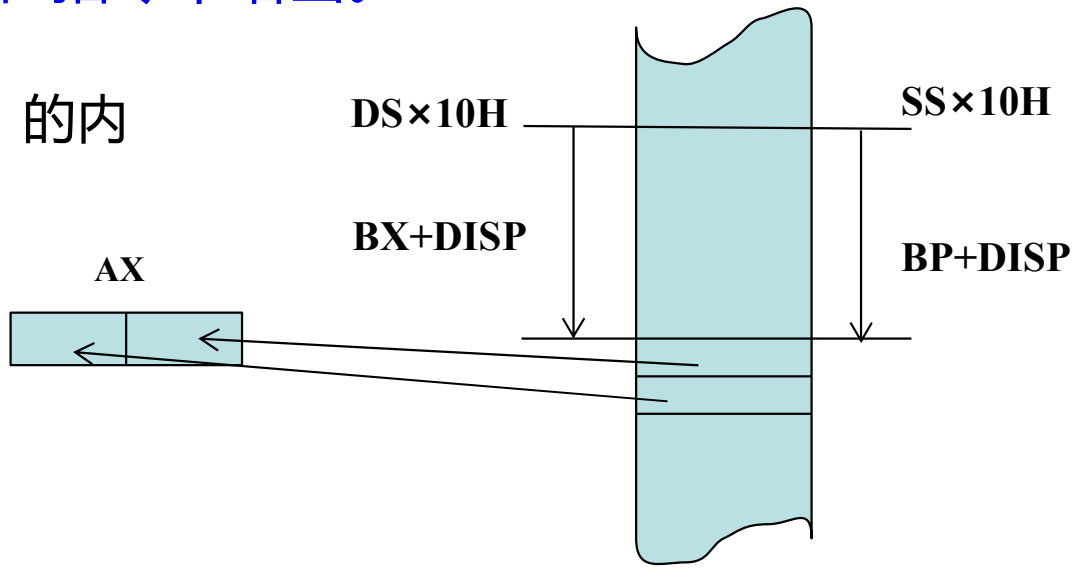
有效地址EA为基址寄存器（BX，BP）的内容加偏移量。

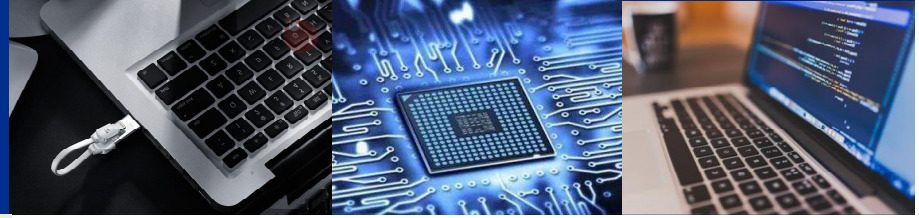
当EA = （BX） + DISP时：

PA = （DS） ×16 + EA

当EA = （BP） + DISP时：

PA = （SS） ×16 + EA





3.2 8086寻址方式

◆偏移量是符号数，

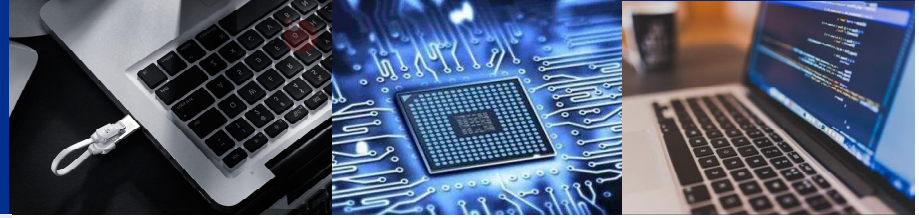
8位偏移量的取值范围为：**0 - 0FFH**（即+127 ~ -128）

16位偏移量的范围：**0 ~ 0FFFFH**（即+32767 ~ -32768）

◆IBM汇编允许用三种形式表示相对寻址，

它们的效果是一样的，如：

```
MOV  AX，[ BX ]+6      ；标准格式
MOV  AX，6 [ BX ]      ；先写偏移值
MOV  AX，[ BX+6 ]      ；偏移值写在括号内
```



3.2 8086寻址方式

④ 基址加变址寻址

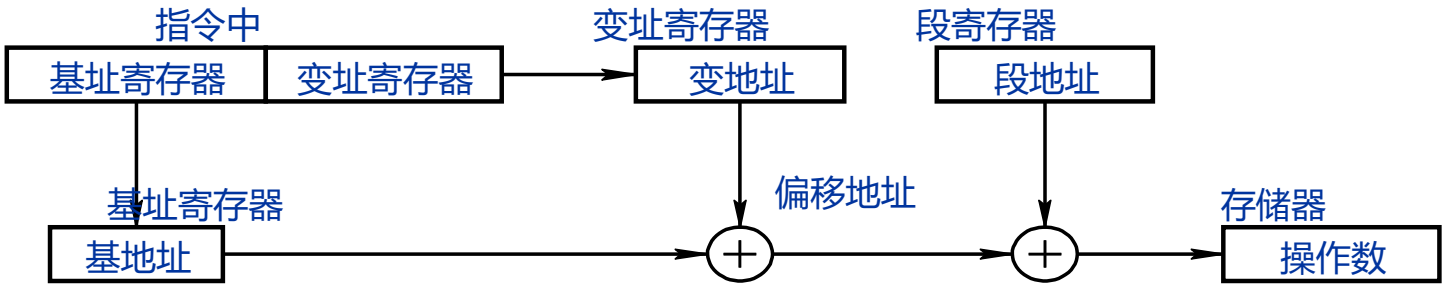
☆ 基址变址寻址时操作数的 EA 分为两部分，分别存于两个寄存器中：

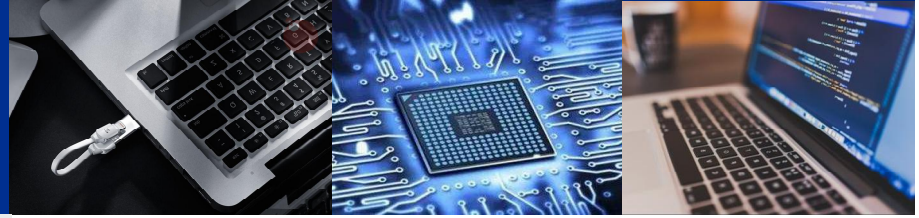
▲ 一部分存于**基址寄存器(BX或BP)**中；

▲ 另一部分存于**变址寄存器(SI或DI)**中；

▲ 指令中分别给出两个寄存器名。操作数的有效地址为：

EA1 = BX+SI / DI　　或　　EA2 = BP + SI / DI





3.2 8086寻址方式

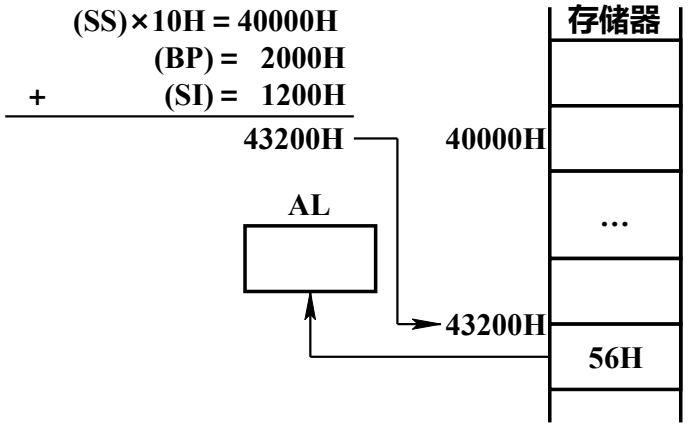
【例】 MOV AL，[BP] [SI]

设：SS =4000H，BP =2000H，SI =1200H，

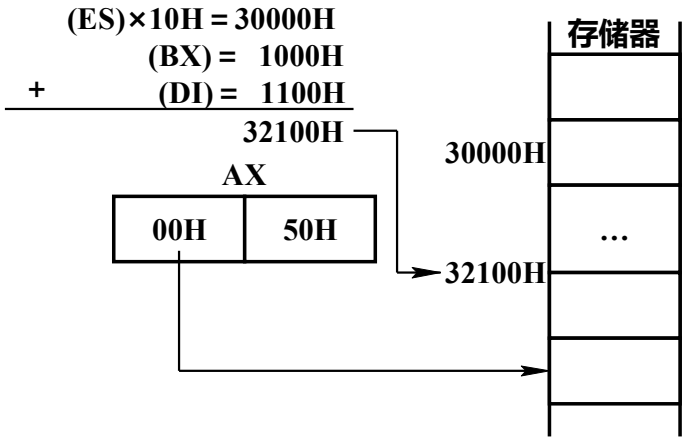
源操作数的寻址过程如图所示

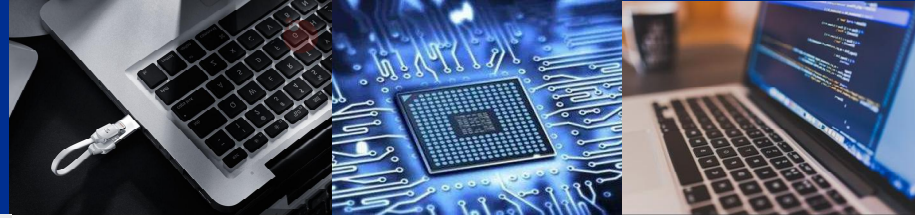
① 根据指令中给出的寄存器名及寄存器内容，得到存储单元的PA为： SS ×16 + BP + SI = 43200H

② 把该内存单元中的内容送到AL中



MOV ES：[BX] [DI],AX





3.2 8086寻址方式

总结：有效地址EA为基址寄存器（BX，BP）加变址寄存器（SI，DI）的内容。

$$EA = \begin{pmatrix} BX \\ BP \end{pmatrix} + \begin{pmatrix} SI \\ DI \end{pmatrix}$$

PA = （DS）×16 + EA，当基址寄存器为BX时

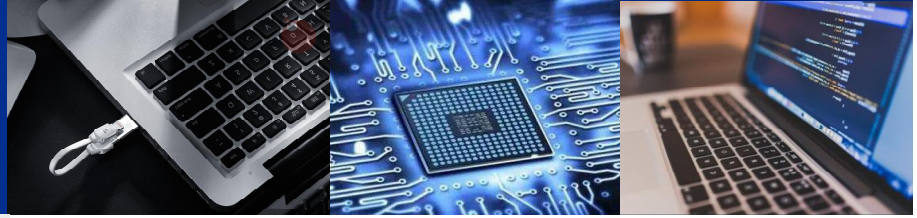
PA = （SS）×16 + EA，当基址寄存器为BP时

MOV [BX+DI],AX ；**目的操作数在数据段** EA = （BX） + （DI），PA = （DS）×16 + EA

MOV DX,[BX][SI] ；**源操作数在数据段** EA = （BX） + （SI），PA = （DS）×16 + EA

MOV [BP][DI],BX ；**目的操作数在堆栈段** EA = （BP） + （DI），PA = （SS）×16 + EA

MOV CX,[BP+SI] ；**源操作数在堆栈段** EA = （BP） + （SI），PA = （SS）×16 + EA



3.2 8086寻址方式

⑤相对基址加变址寻址

☆ 采用相对基址变址寻址时，操作数的有效地址分为三部分：

- ▲ 一部分存于**变址寄存器 SI 或 DI** 中；
- ▲ 一部分存于**基址寄存器 BX 或 BP** 中；
- ▲ 一部分为**偏移量**。
- ▲ 指令中分别给出两个寄存器名及8位或16位的偏移量。

操作数的有效地址EA为：

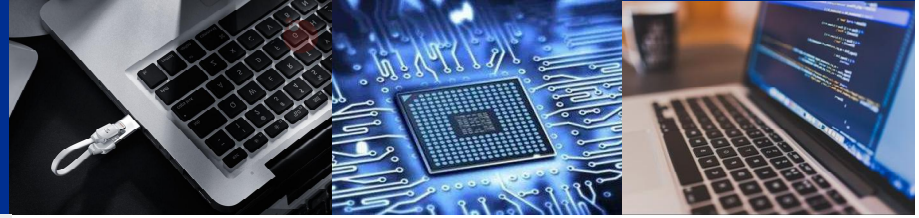
☆ 当基址寄存器选用BX时，数据隐含存于数据段中；

☆ 当基址寄存器选用BP时，数据隐含存于堆栈段中，

操作数的**物理地址PA**为：

$$PA1 = DS \times 10H + EA1$$

$$PA2 = SS \times 10H + EA2$$



3.2 8086寻址方式

【例】　　MOV　DATA　[BP] [SI], CX

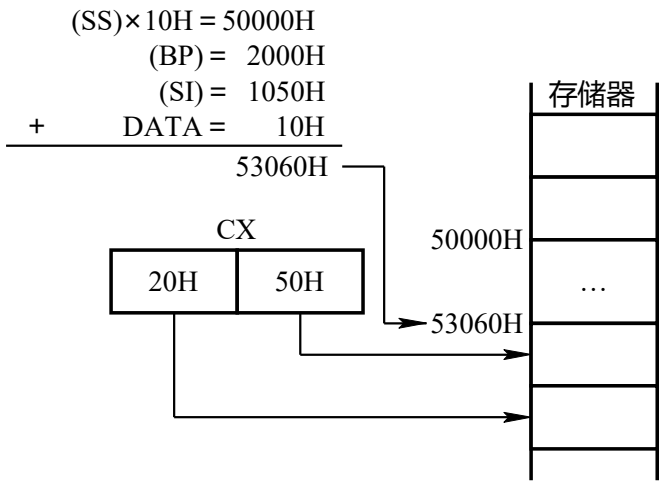
　　等同于　MOV　10H　[BP] [SI], CX

★　相对基址变址寻址的寻址过程：

　　设：　SS　= 5000H；

　　BP　= 2000H；　SI　= 1050H

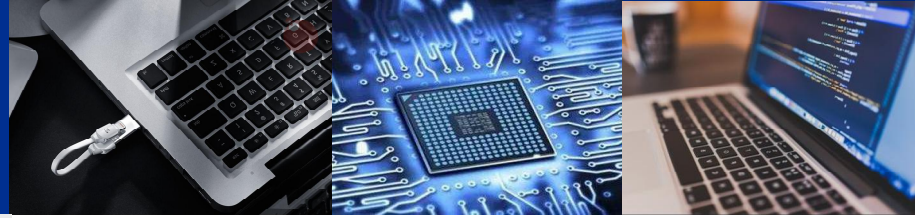
　　源操作数的寻址过程如图所示。



① 由指令中给出寄存器名、寄存器内容及偏移量，得到存储单元的物理地址：

　　SS ×10H ＋BP＋SI＋10H = 53060H

② 把寄存器CX中的内容送到该地址内存单元中。



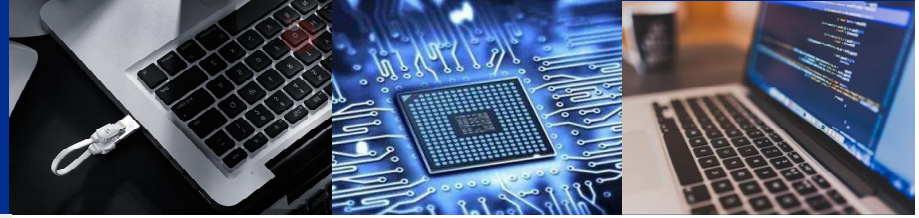
3.2 8086寻址方式

4. I/O端口寻址（I/O Port Addressing）

- I/O接口电路由接口寄存器组成，需要用编号区别各个寄存器：

  编号 = 地址

- **I/O地址**是接口电路中寄存器的编号
- **端口**是I/O地址的通俗说法
- 系统通过这些**端口与外设**进行通信
- 采用十六进制数来表达端口
  - Intel 8086支持**64K个8位端口**
  - I/O地址可以表示为：0000H ~ FFFFH
  - PC机支持1K个8位端口
  - I/O地址可以表示为：000H ~ 3FFH

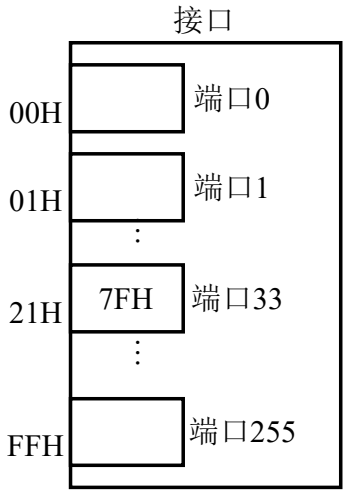


3.2 8086寻址方式

(1) 端口直接寻址方式

端口地址用8位立即数(0～255)表示。

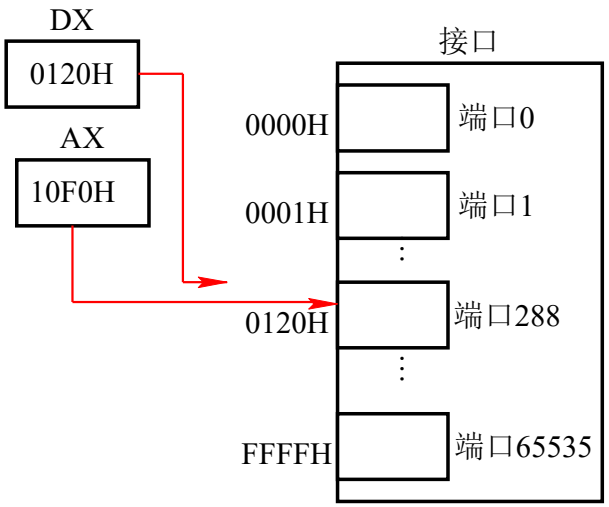
例如：　　IN AL，21H

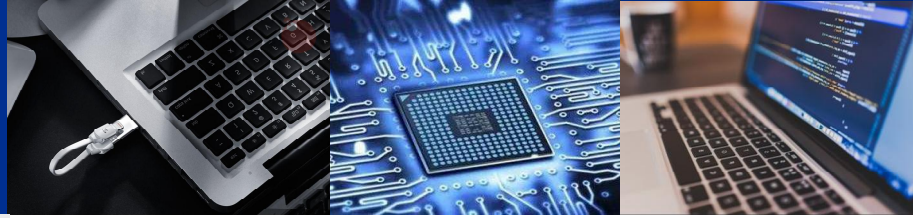


(2) 端口间接寻址方式

当I/O端口地址大于FFH时，必须事先将端口地址存放在DX寄存器中。

例如：　MOV DX，120H
　　OUT DX，AX





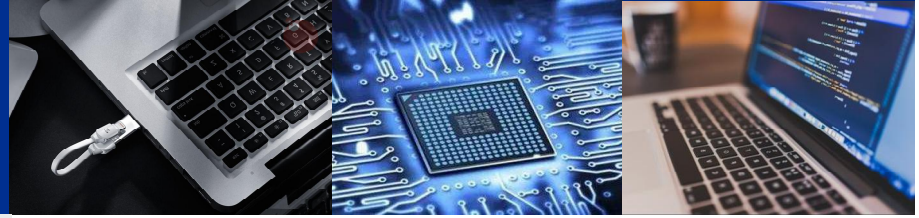
3.2 8086寻址方式

3.2.2 指令寻址方式

- 程序的执行，一般是从存储器MEM中代码段 顺序取指令执行；指令地址是由指令指针寄存器IP自动增量形成。
- 在某些条件下，程序的执行要跳转到另外一段程序开始顺序执行。

这种运行方式称为：**转移。**转移指令中应给出转移地址操作数。

- 它也可以分为：立即数、寄存器操作数、存储器操作数；即要转移的目标地址包含在指令中或存放在寄存器、内存储器中。
- 根据跳转的距离远近，转移又分为：**段内转移 和 段间转移。**
- **段内转移：**转移的目标地址在当前的代码段内。转移指令中要指出转到的目标地址，CPU根据转移指令中指明的目标地址，修改 IP 的值，即修改指令指针，从新址开始执行
- **段间转移：**转移的目标地址在另一个代码段内，转移指令中要指出 转到的代码段的段基址及在该段内的目标地址，CPU据此修改 CS 和 IP 中的值，从而使 CPU 转到另一个代码段中取指令并执行程序。



3.2 8086寻址方式

- **段内转移的寻址方式：**

① **段内直接寻址**：在转移指令中直接给出 8 位或 16 位的相对位移量，指令转向的有效地址为:

(当前 IP 的内容 +相对位移量) → (IP)

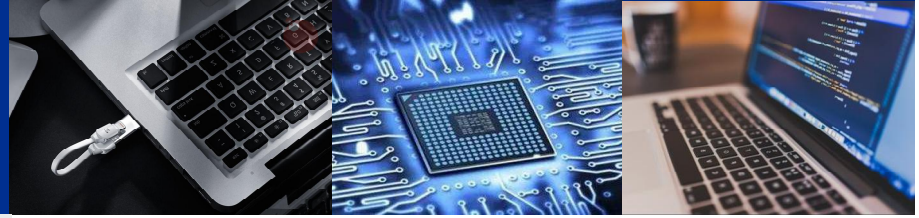
位移量为 8 位时，称段内**直接短转移**；位移量为 16 位时，称段内直接近转移

② **段内间接寻址**：由转移指令中指定一个16位寄存器或存储器字单元的内容做为转移的有效地址，直接取代（IP）的内容。

- **段间转移的寻址方式：**

① **段间直接寻址**：转移指令中直接给出了转向目标的段基址和段内偏移地址，用此地址分别取代（CS）和（IP）中的内容，完成从当前段向另一个段的转移

② **段间间接寻址**：在转移指令中给出一个存储单元的地址，用该地址所指的两个相邻字单元的内容（32位）来取代（CS）和（IP）中的内容，从而达到段间转移的目的。



3.2 8086寻址方式

SHORT：表示位移量在-128~127字节之间，称为短转移，位移量用一个 字节（8位）来表示。

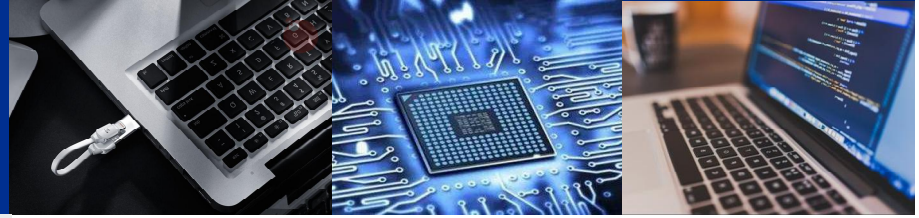
FAR：表示转移距离超过±32K字节，或是在不同段之间转移，称为远转移，因为程序控制超出了当前代码段，所以CS和IP都必须修改为新的值。

NEAR： 表示在同一段内转移，位移量在-32768~32767字节范围内，称为近转移，位移量用16位表示， 因为程序控制仍然在当前代码段，所以只修改IP的值，CS的值不变。

例：段内直接寻址方式

```
                1060:000D EB04 JMP SHORT NEXT
IP当前值→ 1060:000F … …
                1060:0011 … …
                1060:0013 0207 NEXT: ADD AL,[BX]
```

- ✓ CPU在执行JMP指令时，IP指向了下一条指令，其值为000F， JMP SHORT NEXT指令的机器语言为EB04，EB为操作码，04为位移量，
  所以转向的有效地址应为：**000F + 0004 = 0013**
- ✓ **0013正是标号NEXT的地址。**JMP指令执行后，将IP寄存器修改为0013，代码段寄存器CS不变。
  紧接着CPU根据CS:IP的指示，取出1060:0013中的ADD指令开始执行，这样实现了程序的转移。



3.2 8086寻址方式

例：**段内间接寻址方式**

假设（DS）= 2000H，（BX）= 1256H，（SI）= 528FH，（264E5H）= 2450H。

JMP BX；　　则执行该指令后（IP）= 1256H

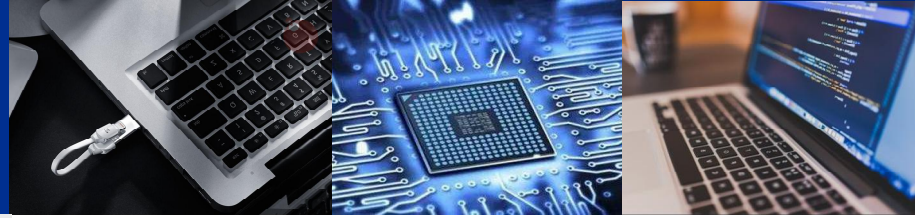
JMP [BX][SI]；　　则指令执行后（IP）=（16d ×（DS）+（BX）+（SI））
=（20000H + 1256H + 528FH）
=（264E5H）= 2450H

例：段间直接寻址方式

JMP **FAR** PTR OUTSEG

因为是段间转移，CS和IP都要更新，这个新的段地址和偏移地址由**OUTSEG**，只要将指令中OUTSEG的**低字装入IP，高字装入CS**，就完成了从一个段到另一个段转移的工作。

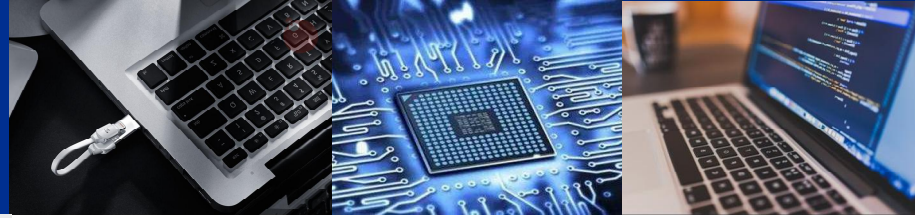
```
JMP    DWORD   PTR [SI]          ;  (IP) = (16d × (DS) + (SI) )
                                  ;  (CS) = (16d × (DS) + (SI) +2)
JMP    DWORD   PTR [TABLE+BX] ;  (IP) = (16d × (DS) + (BX) + TABLE )
                                  ;  (CS) = (16d × (DS) + (BX) + TABLE +2)
```



3.2 8086寻址方式

段寄存器的使用规定

访问存储器的方式	默认的段寄存器	可跨越的段寄存器	偏移地址
取指令	CS	无	IP
堆栈操作	SS	无	SP
一般数据访问	DS	CS ES SS	有效地址EA
BP作为基址的寻址	SS	CS DS ES	BP
串操作的源操作数	DS	CS ES SS	SI
串操作的目的操作数	ES	无	DI



3.3 8086指令系统

拥有138条基本指令，按功能分成六类：

（1）数据传送指令

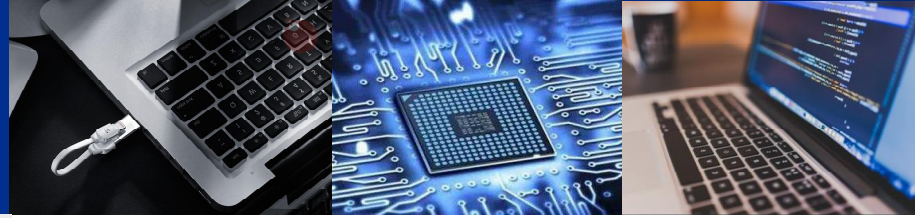
（2）算术运算指令

（3）逻辑运算指令

（4）串操作指令

（5）控制转移指令

（6）处理器控制指令



3.3 8086指令系统

3.3.1 数据传送指令

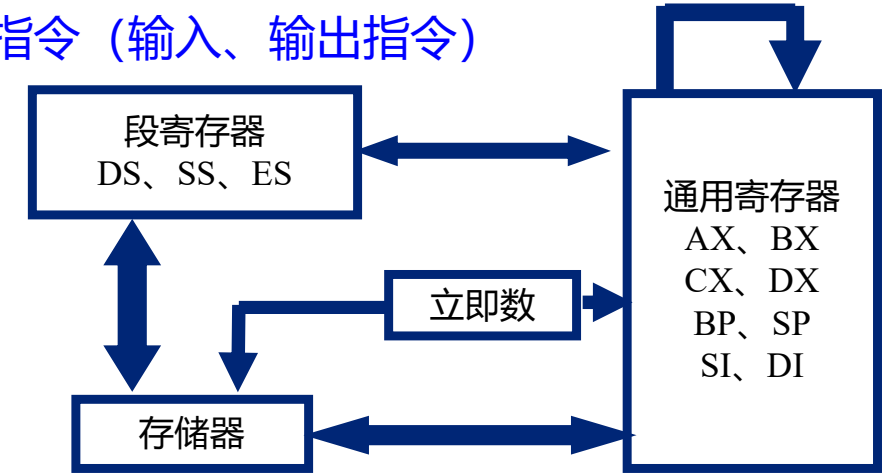
数据传送：负责把数据、地址或立即数送到寄存器或存储单元中。

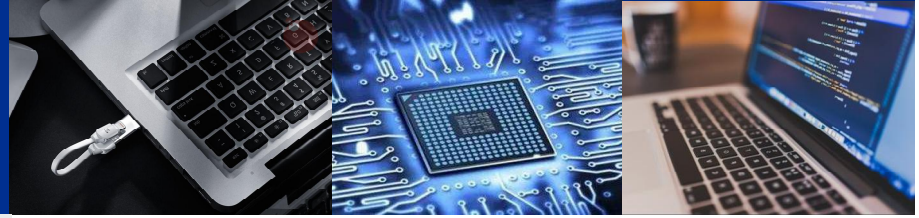
分为四个类型：
① 通用数据传送指令
② 地址传送指令
③ 标志位传送指令
④ 专用累加器传送指令（输入、输出指令）

1. 传送字或字节指令 MOV

格式：MOV dst, src ;

功能：src → dst





3.3 8086指令系统

MOV指令的特点

- 这里的传送实际上是**复制**，把src的内容复制给dst，src内容不变；
- src和dst**必须类型一致**（都是8位或者是16位）；
- dst**不能是立即数；**
- 两个操作数**不能都是存储器操作数；**
- 两个操作数**不能都是段寄存器操作数**；
- **src是立即数时**，dst不能是段寄存器，必须通过通用寄存器作中介；

(1) 立即数传送给通用寄存器或存储器

```
MOV AL,12H
; 8位数据传送，将12H传送到寄存器AL中
MOV AX,3456H
; 16位数据传送，将3456H传送到寄存器AX中
```

(2) 通用寄存器之间相互传送

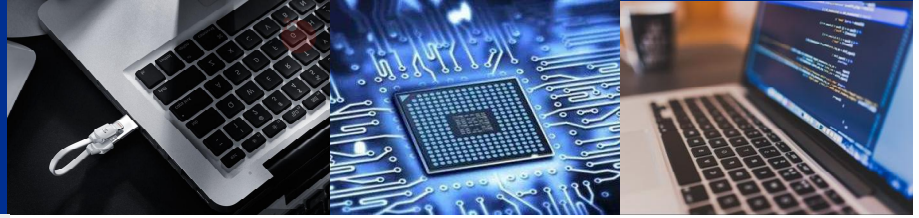
```
MOV AX,BX
; 16位数据传送，将BX中的数据传送到寄存器AX中
MOV CL,BH
; 8位数据传送，将BH中的数据传送到寄存器CL中
```

(3) 通用寄存器和存储器之间相互传送

```
MOV AX,［BX］
; 16位数据传送，将BX指定的连续2个字节中的数据传送到AX中
MOV［SI］,DH
; 8位数据传送，将DH中的数据传送到由SI指定的内存单元中
```

(4) 段寄存器与通用寄存器、存储器之间的相互传送

```
MOV DS,AX
MOV BX,ES
MOV ES,［SI］
MOV［DI］,SS
```



3.3 8086指令系统

- **CS和IP不能作为目的操作数，CS可以作为源操作数。**
- CS 和 IP的值只在**控制转移指令**中修改。
- 对**非控制转移指令**，取完指令后**IP值自动** 指向下条指令。
- 段寄存器CS的值，只在**MOV、PUSH**中可作操作数，**--源操作数**

且这两条指令执行结果不改变CS值。

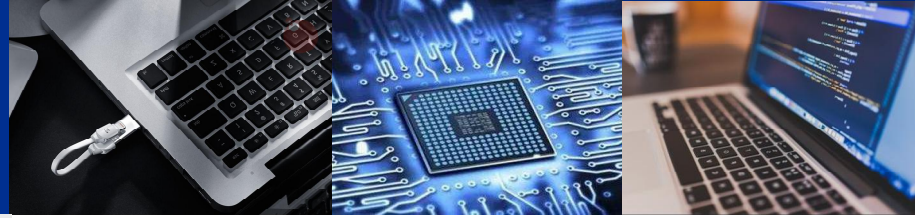
```
MOV    AX, CS
PUSH   CS
```

IP、FLAGS两个寄存器不作为操作数在指令中出现。

```
MOV   IP ,    1234H
MOV   FLAGS , 0F0FH
```

- FLAGS状态寄存器的状态位值**由指令执行后确定，**

不同的指令对各标志的影响不同。



3.3 8086指令系统

2. 堆栈操作指令

入栈格式：PUSH　src ;　　src: { MEM，REG，SEG }

功能：将src指示的字数据压入当前栈顶，IP除外

出栈格式：POP　dst ;　　dst: { MEM，REG，SEG

功能：将当前栈顶的字弹出到 dst 中，CS除外

堆栈是一个**“先进后出”**的主存区域，位于堆栈段中，使用SS段寄存器记录其段地址。栈只有一个出口，即当前栈顶。
栈顶是地址较小的一端（低端），它用堆栈指针寄存器SP指定。

注意几点：

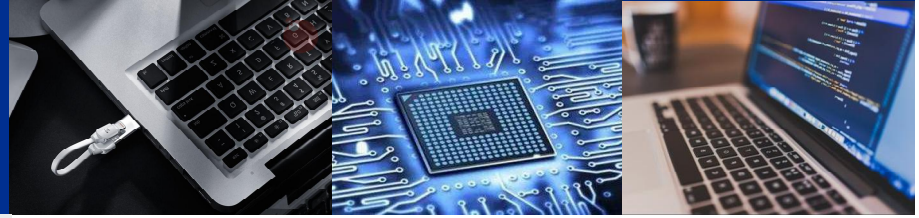
（1）因为堆栈指针**SP**总是指向已经存入数据的**栈顶**（不是空单元），所以PUSH指令是先将（SP）减2，后将内容压栈（即先修改SP使之指向空单元，后压入数据），而POP是先从栈顶弹出一个字，后将堆栈指针SP加2。

（2）**PUSH CS是合法的**，但是**POP CS是不合法的**。

（3）因为SP总是指向栈顶，而用PUSH和POP指令存取数时都是在栈顶进行的，所以堆栈是“先进后出”或叫“后进先出”的。栈底在高地址，堆栈是从高地址向低地址延伸的，所以栈底就是最初的栈顶。

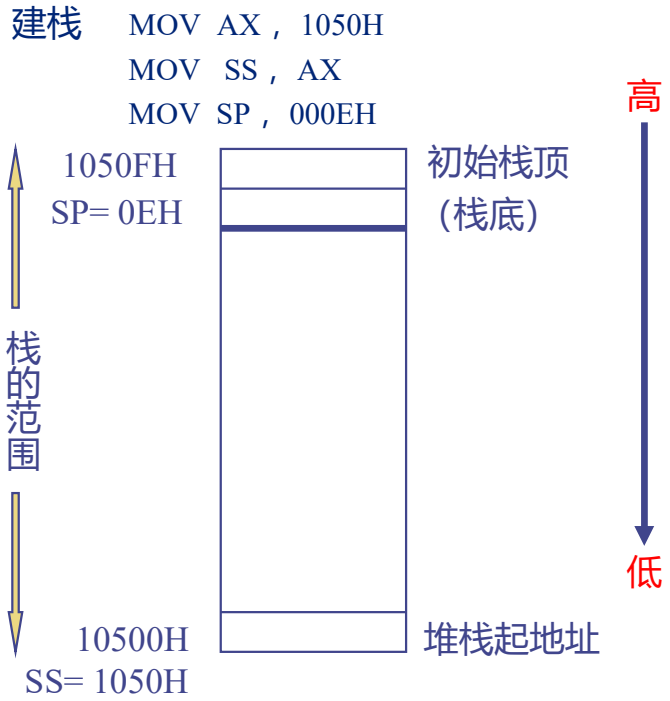
（4）用PUSH指令和POP指令时只能按字访问堆栈，不能按字节访问堆栈。

（5）PUSH和POP指令**都不影响标志**。



3.3 8086指令系统

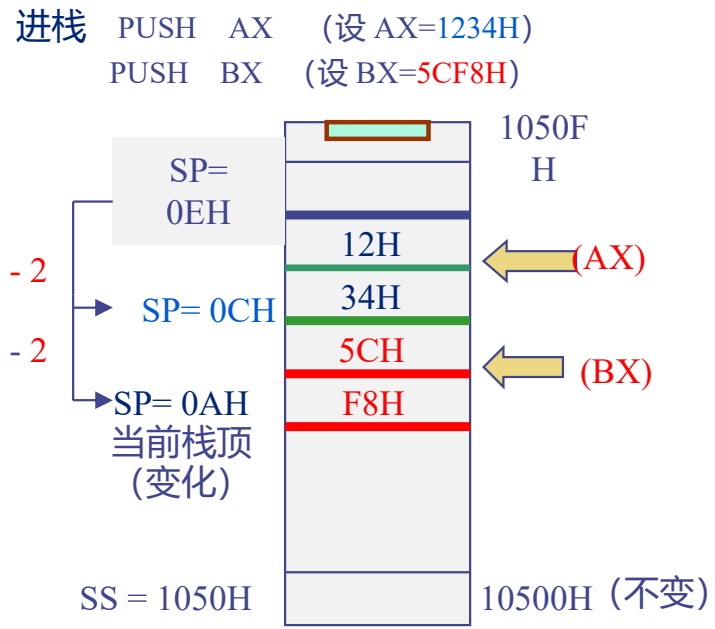
8086通过赋值SS和SP建栈



进栈前SP和SS的值

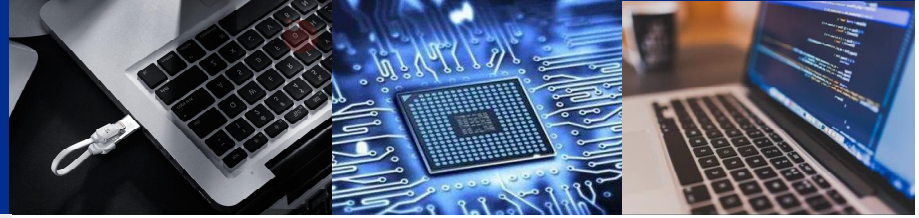
SP=000EH SS=1050H

进栈操作：先SP减 2，再内容进栈。



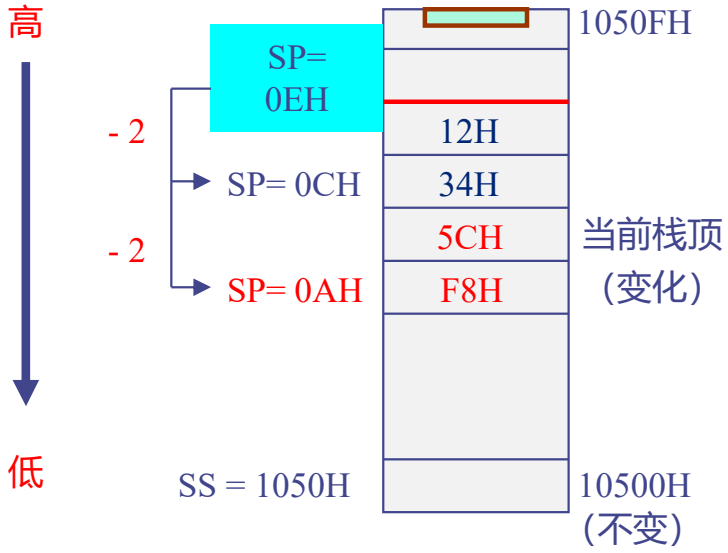
进栈后的SP和SS的值

SP=000AH SS=1050H



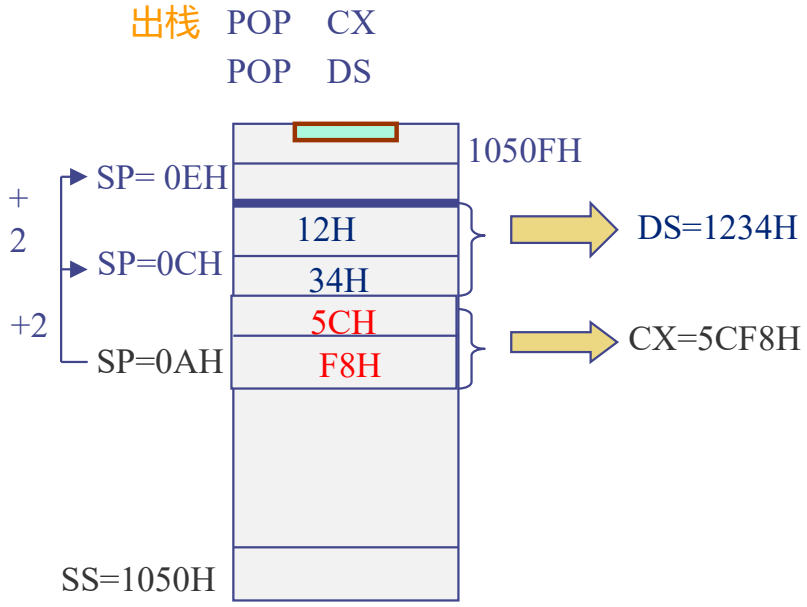
3.3 8086指令系统

出栈操作：先栈顶内容出栈，再修改SP，使SP加2。 -（字操作）



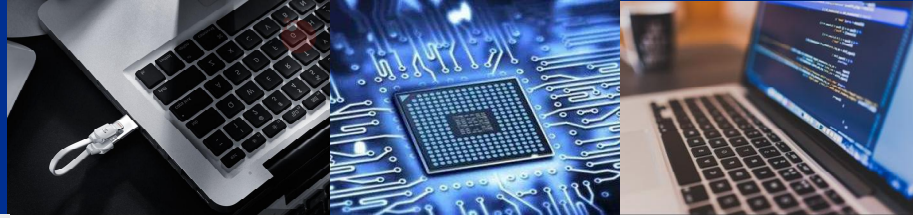
出栈前的SP和SS的值

SP=000AH SS=1050H



出栈后SP和SS的值

SP=000EH SS=1050H



3.3 8086指令系统

3. 地址传送指令

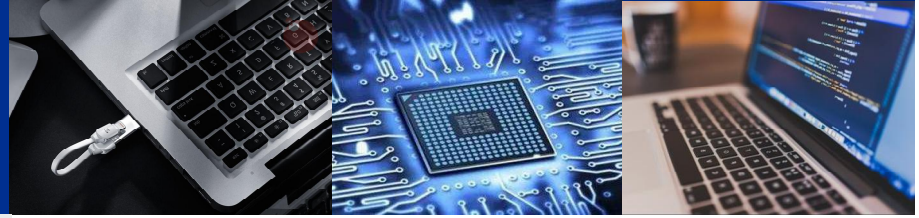
① 有效地址EA传送指令 LEA

格式：LEA　REG，MEM ；

功能：将**MEM**操作数的有效地址**EA**送到寄存器**REG**中

- 例子：LEA BX，[BX+DI+2000H]
  - 指令执行前：BX=4000H，DI=0100H
  - 指令执行后：BX=4000+0100+2000H=6100H

- 注意：
  - 源操作数必须是存储器操作数，而目的操作数必须是16位通用寄存器。
  - 不影响标志位



3.3 8086指令系统

② 数据段寄存器传送指令 LDS

格式：LDS　REG，MEM ；　{ [MEM] → REG ; [MEM + 2] → DS }

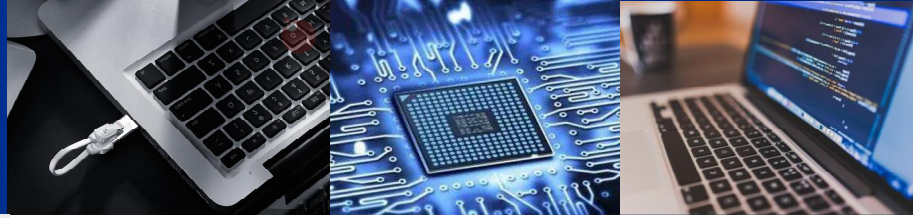
功能：从源操作数所指定的存储器单元中取出4个字节的变量地址指针，把前两个字节（变量的偏移地址）传送到目标操作数，后两个字节（变量的段基址）传送到DS段寄存器中。

例子：LDS SI，[1000H]

- 指令执行前：DS=6000H，（61000H）=0600H，（61002H）=2000H
- 指令执行后：SI=0600H，DS=2000H

◆ **注意**：

- 变量的16位地址偏移量必须传送至一个16位的通用寄存器，典型为SI。
- 不影响标志位



3.3 8086指令系统

③ 附加段寄存器传送指令 LES

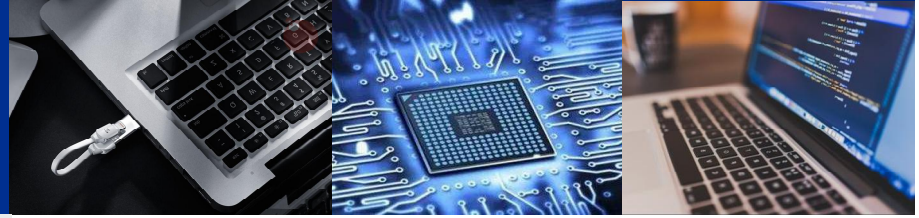
格式：LES　REG，MEM ；　[MEM] → REG
　　[MEM + 2] → ES

功能：从源操作数所指定的存储器单元中取出4个字节的变量地址指针，把前两个字节（变量的偏移地址）传送到目的操作数，后两个字节（变量的段基址）传送到ES段寄存器中。

例：设某双字存储单元偏移地址为3000H，双字数据为12345678H，则

LDS　SI，[3000H]　；DS=1234H，SI=5678H

LES　DI，[3000H]　；ES=1234H，DI=5678H



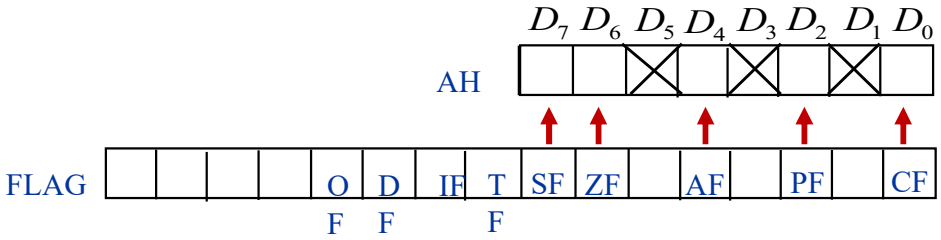
3.3 8086指令系统

4. 标志寄存器传送指令

①FR 寄存器低八位传送至AH指令 LAHF

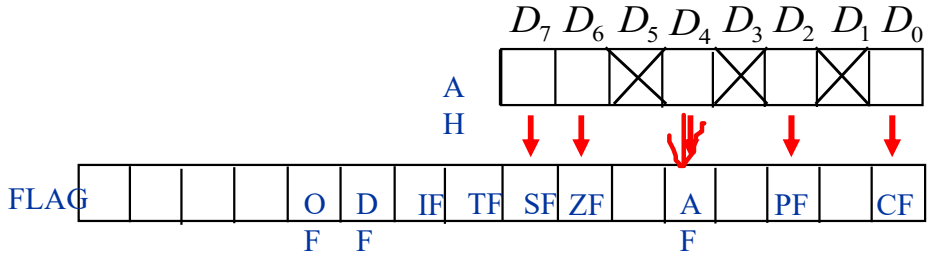
格式：LAHF ；

FR-L → AH



② AH内容送FR 寄存器低八位指令 SAHF

格式：SAHF ；　　AH → FR-L



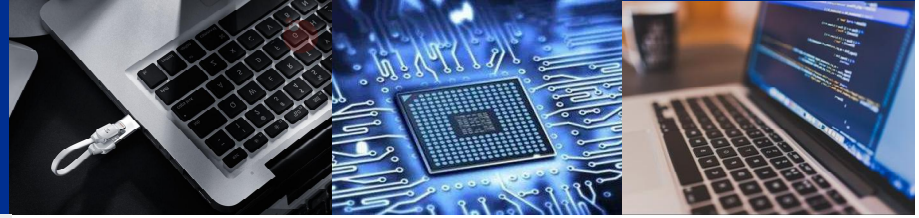
③ FR寄存器 压栈指令 PUSHF

格式：PUSHF ；　　FR → [SS×16 + SP -2]

④ FR寄存器 出栈指令 POPF

格式：POPF ；　　FR → [SS×16 + SP]

SP + 2 → SP



3.3 8086指令系统

5. 数据交换指令

操作数：两操作数可以是通用寄存器和存储器。

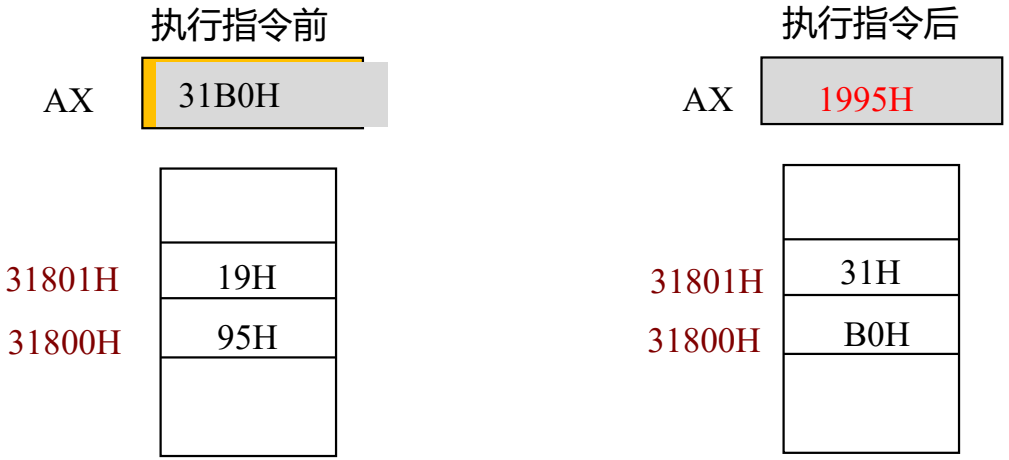
注意点：两操作数不能都是存储器操作数；

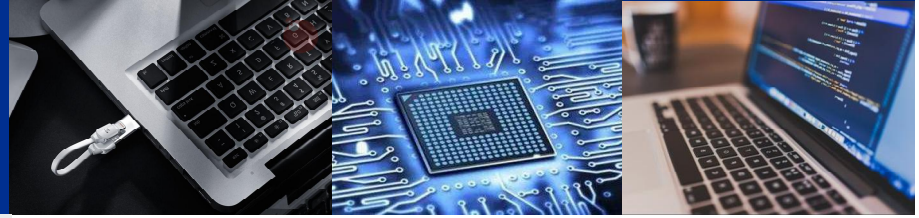
两操作数类型必须一致。

格式：XCHG　OPR1，OPR2；

功能：将两个操作数（字或字节）相互交换位置

例：设AX=31B0H，DS=3000H, BX=1800H，(31800H) = 1995H，执行指令 XCHG　AX, [BX] 的过程为：





3.3 8086指令系统

6. 换码指令XLAT

执行的操作：**AL←[(BX)+(AL)]**

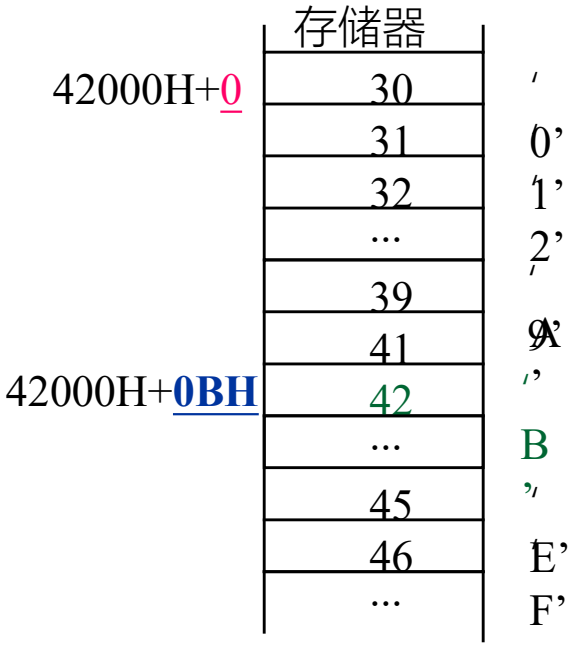
又叫**查表转换指令**，它可根据表项序号查出表中对应代码的内容。执行时先将表的首地址（偏移地址）送到BX中，表项序号存于AL中。

例如：内存数据段有一张16进制数的ASCII码表，设首地址为2000H，如欲查出表中第11个代码（代码序号从0开始）即十六进制数B的ASCII码。ASCII表在DS段中，并假设(DS)=4000H。

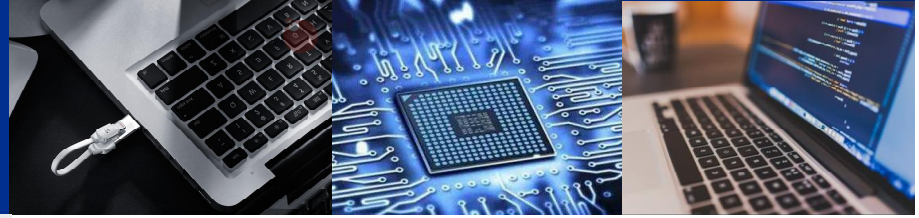
```
MOV  BX，2000H  ；  （BX）←表首地址
MOV  AL，0BH    ；  （AL）←序号
XALT            ；  查表转换
```

执行后得到： （AL）= 42H = ’B’

注意：转换表长度最大为256个表项(字节)。



十六进制数ASCII码表



3.3 8086指令系统

7. 输入/ 输出数据传送指令：输入输出（I/ O ）指令

只能使用 AL 或 AX 寄存器与 I/ O 设备进行数据交换 不影响FR

① **直接寻址I/O指令**：IN AL，PORT； [PORT] → AL 00H-FFH 256个字节

IN AX，PORT； [PORT] → AL，[PORT+1] → AH

OUT PORT，AL； AL → [PORT]

OUT PORT，AX； AL → [PORT]， AH → [PORT+1]

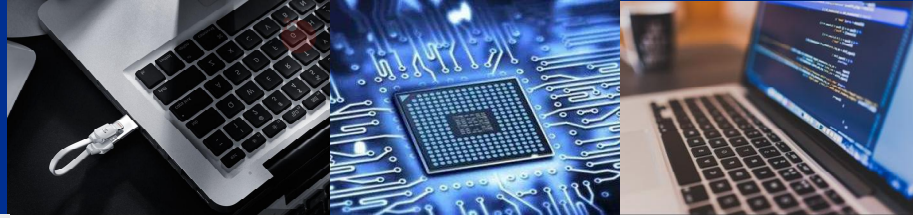
② **间接寻址I/O指令**：IN AL，DX； [DX] → AL 0000H-FFFFH端口号

IN AX，DX； [DX] → AL，[DX+1] → AH

OUT DX，AL； AL → [DX]

OUT DX，AX； AL → [DX]， AH → [DX+1]

- 间接寻址I/O指令必须使用DX为间址寄存器，指明I/O端口地址



3.3 8086指令系统

3.3.2算术运算指令

分为五个类型：
- 加法指令
- 减法指令
- 乘法指令
- 除法指令
- 十进制调整指令

1.加法与减法指令

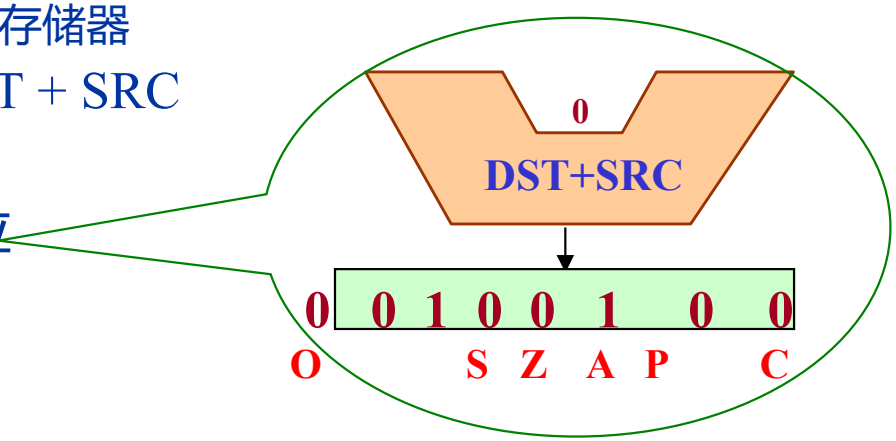
(1) ADD DST, SRC

源操作数：通用寄存器、存储器、立即数

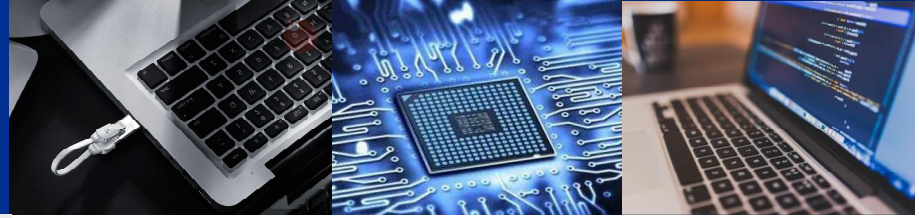
目的操作数：通用寄存器、存储器

执行的操作：DST ← DST + SRC

注：该指令影响标志位



注意： 该指令适合有符号数和无符号数的运算。



3.3 8086指令系统

例： MOV DX, 4652H
ADD DX, 0f0f0H

```
  4652
+ f0f0
------
  3742
1
```

```
  0100011001010010
+ 1111000011110000
------------------
  0011011101000010
```

执行后： (DX) = 3742H

Z=0　　S = 0　　C= 1　　O = 0

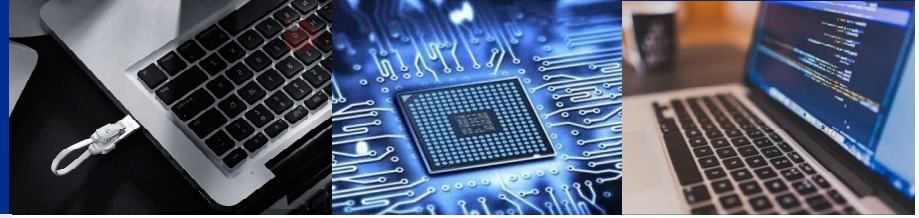
对无符号数　溢出　　结果错

对有符号数　不溢出　　结果正确

问： 对带符号数和无符号数怎样判断是否溢出?

带符号数的溢出用溢出标志O 判断;

无符号数的溢出用进位标志 C 判断。



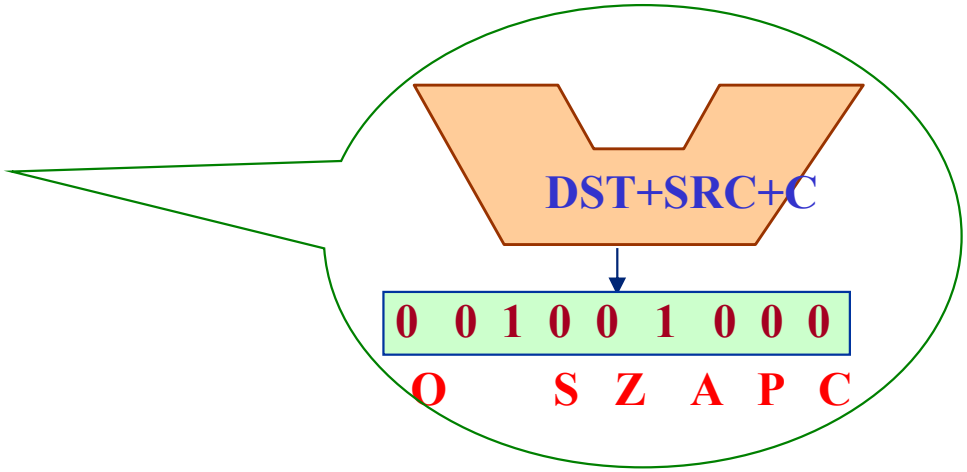
3.3 8086指令系统

(2) ADC DST，SRC

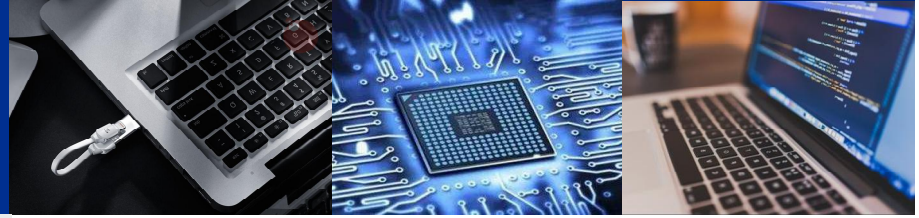
源操作数：通用寄存器、存储器、立即数
目的操作数：通用寄存器、存储器
执行的操作：**DST ← DST + SRC + C**

功能： 主要用于多字节运算

注：该指令影响标志位



注意： 该指令适合有符号数和无符号数的运算。



3.3 8086指令系统

(3) INC OPRD

操作数：通用寄存器、存储器 (不能是段寄存器或立即数)

执行的操作：**OPRD ← OPRD + 1**

功能： 用于在循环中修改地址指针及循环次数等。

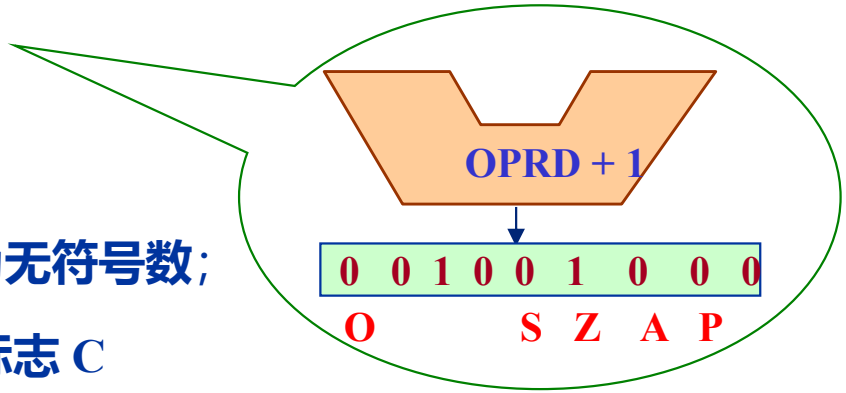
注：该指令影响标志位

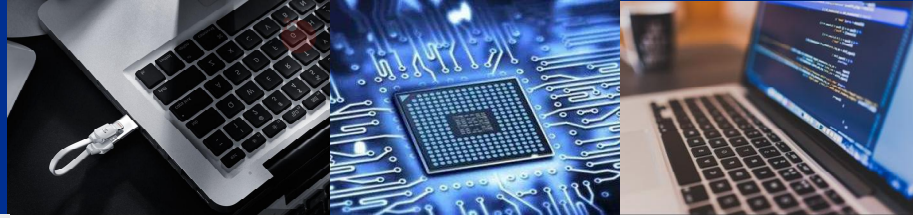
例： INC BX

INC byte ptr [SI]

注意： (1) 该指令将操作数视为**无符号数**；

(2) 该指令不影响 **进位标志 C**





3.3 8086指令系统

(4) 减法指令

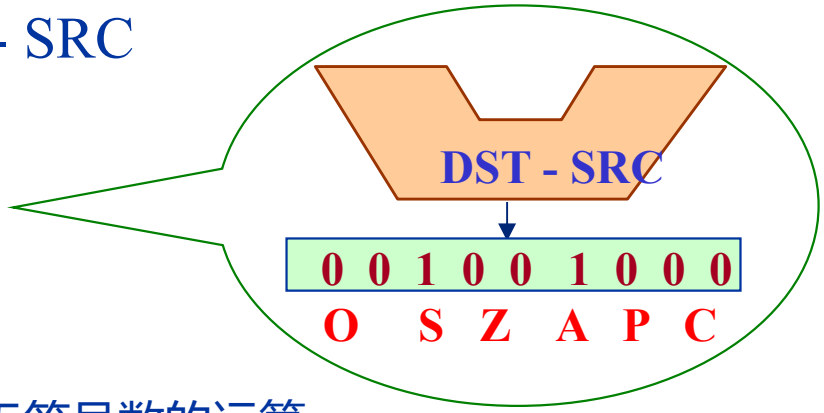
SUB DST, SRC

源操作数：通用寄存器、存储器、立即数

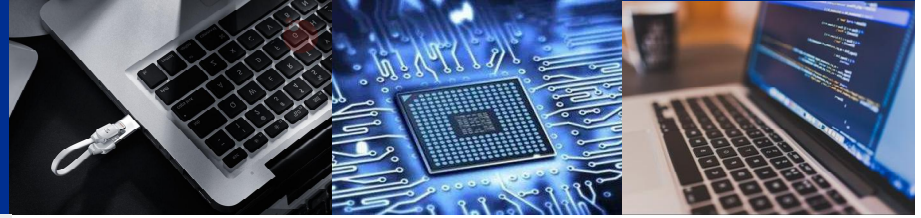
目的操作数：通用寄存器、存储器

执行的操作：DST ← DST - SRC

注：该指令影响标志位



注意： 该指令适合有符号数和无符号数的运算。



3.3 8086指令系统

(5) SBB DST, SRC

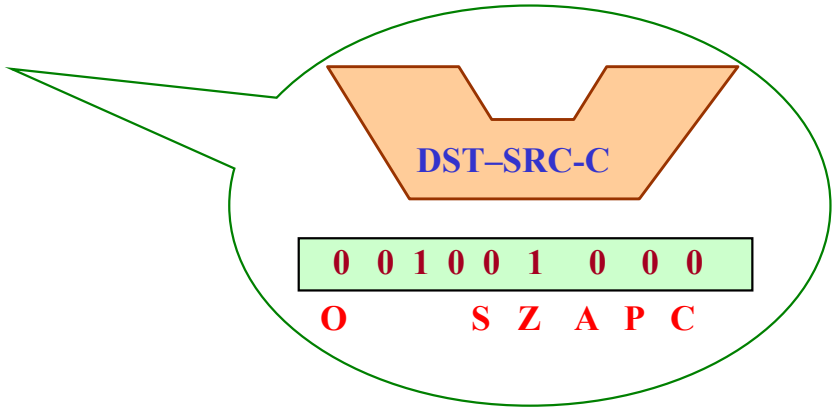
源操作数：通用寄存器、存储器、立即数

目的操作数：通用寄存器、存储器

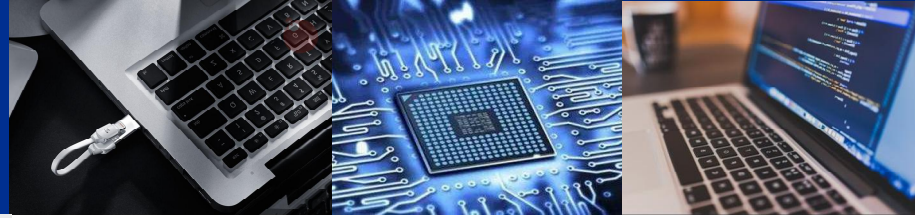
执行的操作：**DST ← DST - SRC –C**

注：该指令影响标志位

功能： 主要用于多字节运算



注意： 该指令适合有符号数和无符号数的运算。



3.3 8086指令系统

注意： 同加法指令　　有符号数的溢出用溢出标志 O 判断；

无符号数的溢出用进位标志 C 判断。

例： SUB DH, [BP+4]　　(DH)=41H　　(SS)= 0000H　　(BP)=00E4H

(00E8H)= 5AH

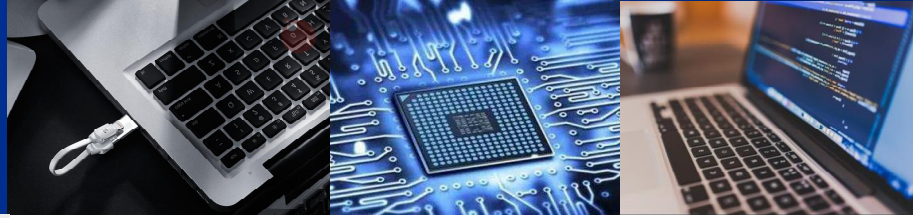
```
  41          01000001
- 5A        - 01011010
--------    ----------
  E7        11100111
 (1)         (1)
```

结果： (DH)=0E7H

S=1　　Z=0　　C=1　　O=0

如果为有符号数则结果正确（无溢出）

如果为无符号数则结果错误（有溢出）



3.3 8086指令系统

(6) DEC OPRD

操作数：通用寄存器、存储器 (不能是段寄存器或立即数)

执行的操作：**OPRD ← OPRD - 1**

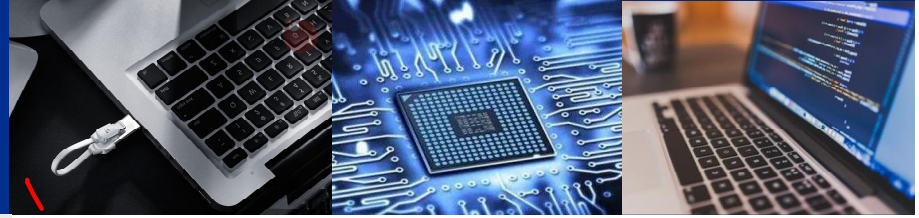
功能： 用于在循环中修改地址指针及循环次数等。

注：该指令影响标志位

例： DEC BX
DEC word ptr[SI]

注意： (1) 该指令将操作数视为**无符号数;**

(2) 该指令不影响 **进位标志 C**



3.3 8086指令系统

(7) NEG OPRD

操作数：通用寄存器、存储器

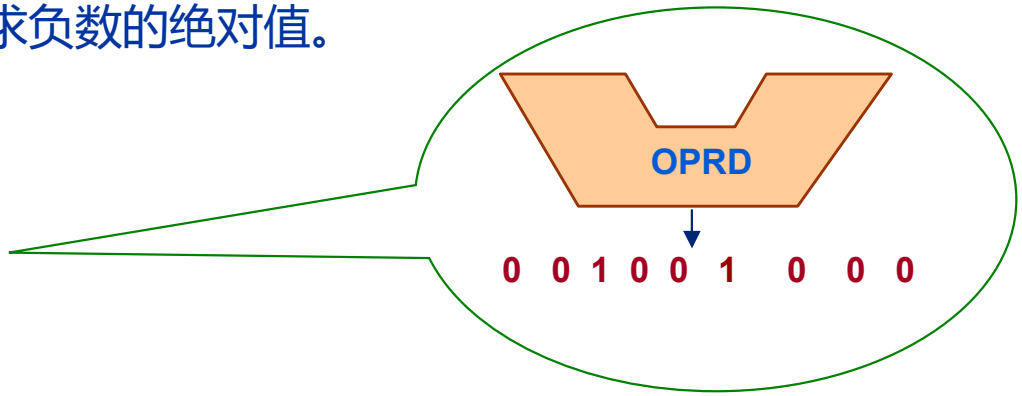
执行的操作：**求补指令，将操作数按位取反后加1，再送回操作数。**

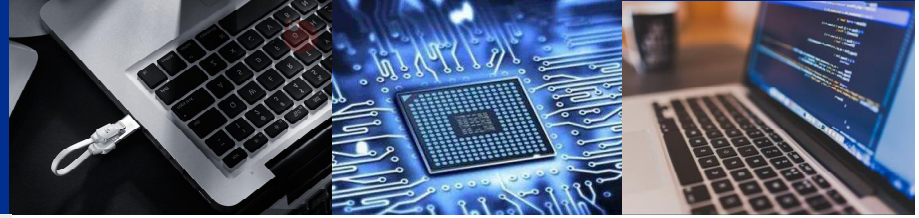
对正数的补码求补变为其负数的补码，

对负数的补码求补变为其正数的补码。

利用NEG指令可以求负数的绝对值。

注：该指令影响标志位



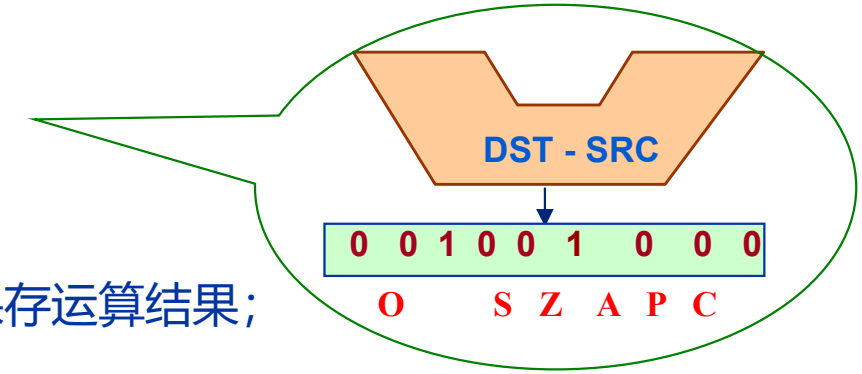


3.3 8086指令系统

(8) CMP DST, SRC

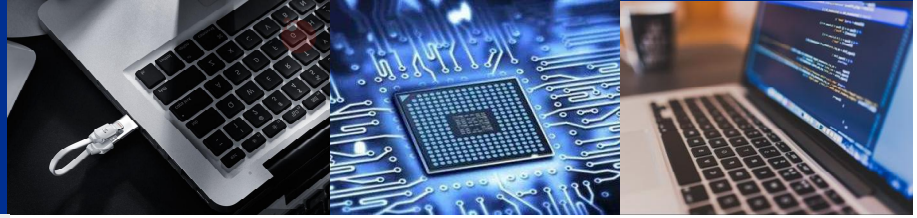
源操作数：通用寄存器、存储器、立即数
目的操作数：通用寄存器、存储器
执行的操作： **DST - SRC**

注：该指令影响标志位



注意: (1) 该指令同 SUB , 但其不保存运算结果；

(2) 该指令后面通常跟一条转移指令，根据标志位产生不同的程序分支。



3.3 8086指令系统

比较指令主要用于两个数之间的关系：大、小、相等。

如：A，B

1. 比较相等。 只要看Z 标志。 Z=1——相等

Z=0——不相等。

2. 比较大小。

分两种情况。

- **无符号数 看进（借）位C 标志。**

A=A0H 1 0 1 0 0 0 0 0

B=53H - 0 1 0 1 0 0 1 1

0 1 0 0 1 1 0 1

A>B

A=53H 0 1 0 1 0 0 1 1

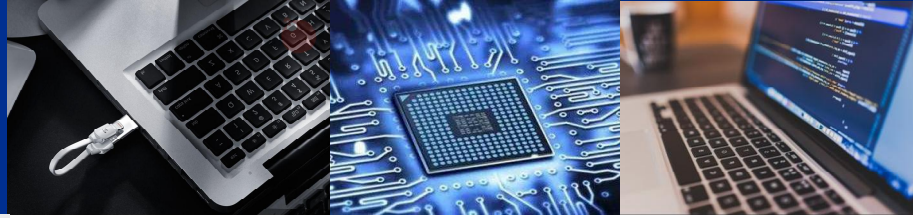
B=A0H - 1 0 1 0 0 0 0 0

1 0 0 1 1 0 0 1 1

A<B

结论：C=1 A<B

C=0 A>B



3.3 8086指令系统

- 带符号数

  * 同号：不会溢出， O=0， 用符号位 S 判断

```
A=24H      0 0 1 0 0 1 0 0
B=44H    - 0 1 0 0 0 1 0 0        O=0    S=0    A>B
           1 1 1 0 0 0 0 0               S=1    A<B
```

  * 异号：可能会溢出，用溢出位O和符号位S 判断

有溢出：

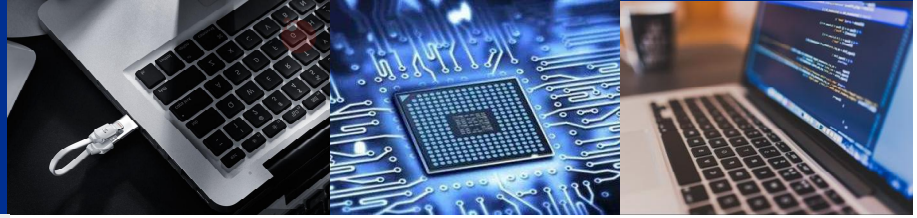
```
A=7FH      0 1 1 1 1 1 1 1
B=F0H    - 1 1 1 1 0 0 0 0        O=1    S=1    A>B
           1 0 0 0 1 1 1 1               S=0    A<B
```

无溢出:

```
A=3FH      0 0 1 1 1 1 1 1
B=F0H    - 1 1 1 1 0 0 0 0        O=0    S=0    A>B
           0 1 0 0 1 1 1 1               S=1    A<B
```

结论： $O \oplus S = 0$ A>B

$O \oplus S = 1$ A<B



3.3 8086指令系统

在分支程序设计中，常用CMP指令来产生条件，其后往往跟着一条条件转移指令，由CMP指令为条件转移指令提供控制转移的依据。

有符号数比较条件转移指令:

JG 标号地址； dst > src

JL 标号地址； dst < src

JGE 标号地址； dst ≥ src

JLE 标号地址； dst ≤ src

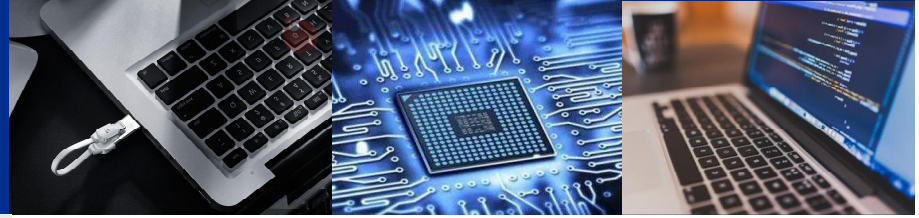
无符号数比较条件转移指令:

JA 标号地址； dst > src

JB 标号地址； dst < src

JAE 标号地址； dst ≥ src

JBE 标号地址； dst ≤ src



3.3 8086指令系统

2. 乘法指令与除法指令

(1) MUL SRC —— 无符号数乘法

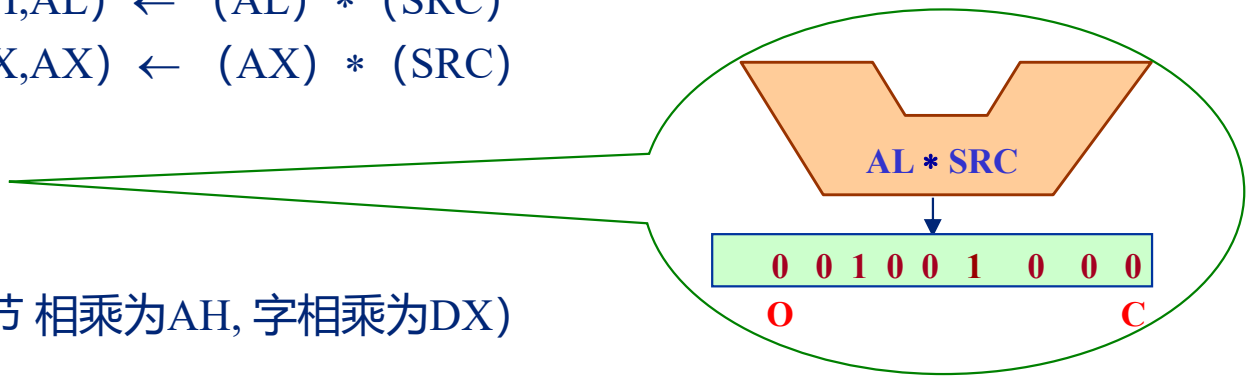
源操作数：通用寄存器、存储器（不能是立即数）

目的操作数：**DX, AX （隐含）**

执行的操作：字节操作 （AH,AL）← （AL）*（SRC）

字操作 （DX,AX）← （AX）*（SRC）

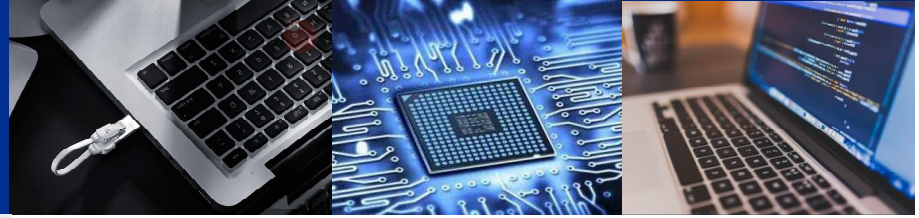
注：**该指令影响标志位**



注意：若结果的高半部分（字节 相乘为AH, 字相乘为DX）

为0 则 C=0，O=0

为1 则 C=1，O=1



3.3 8086指令系统

IMUL SRC —— 带符号数乘法

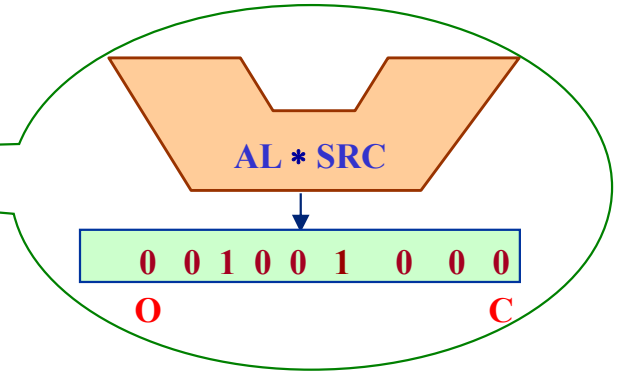
源操作数：通用寄存器、存储器（不能是立即数）

目的操作数：DX, AX （隐含）

执行的操作：字节操作 （AH,AL）← （AL）*（SRC）

字操作 （DX,AX）← （AX）*（SRC）

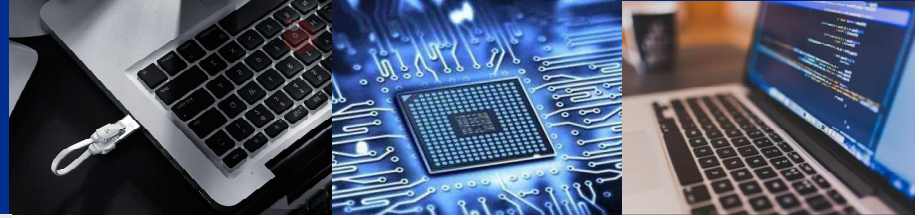
注：该指令影响标志位



注意：若结果的高半部分不是低半部分的符号扩展的话，

则 C=1，O=1

否则 C=0，O=0



3.3 8086指令系统

(2) 除法指令

无符号数除法指令：　DIV SRC

带符号数除法指令：　IDIV SRC

执行操作：

字节操作　(AL) ← (AX) / (SRC) 的商

(AH) ← (AX) / (SRC) 的余数

字操作　(AX) ← (DX, AX) / (SRC) 的商

(DX) ← (DX, AX) / (SRC) 的余数

DIV指令是除法指令，它会把一个寄存器中的值除以另一个寄存器中的值，并将结果存放在被除数寄存器中。如果除数为零，DIV指令会引发一个异常。

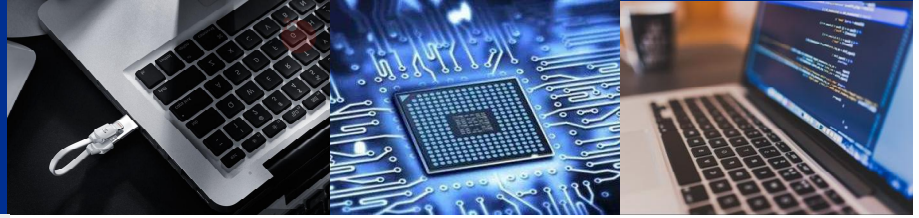
因为DIV指令的运算结果对状态标志无定义，我们不能根据状态标志来判断除法结果的有效性。所以，我们需要通过其他方式来判断。

一种常见的方法是检查除数是否为零。如果除数为零，那么结果就是未定义的，应该抛出异常或返回一个特定的值。

另外，还需要注意DIV指令的一些其他特性。例如，如果被除数是最大的数，而除数小于最大的数，那么结果会是0。如果被除数是负数，而除数是正数，那么结果会是负数。如果被除数是正数，而除数是负数，那么结果会是正数。这些规则可以帮助你判断除法结果的有效性。

总的来说，判断DIV指令的结果是否有效是一个复杂的问题，需要根据具体的应用场景和需求来决定。在处理除法指令时，一定要注意除数为零的情况，避免产生未定义的结果。

注意：* AX (DX,AX) 为隐含的被除数寄存器。
* AL (AX) 为隐含的商寄存器。
* AH (DX) 为隐含的余数寄存器。
* SRC不能为立即数。
* 对所有条件标志位均无定义。如何判别结果有效?



3.3 8086指令系统

3. 符号扩展指令

字节扩展成字指令 CBW

格式：CBW ；

功能：将 AL 中的符号位扩展到 AH 中，

AX 为一个带符号的16位数

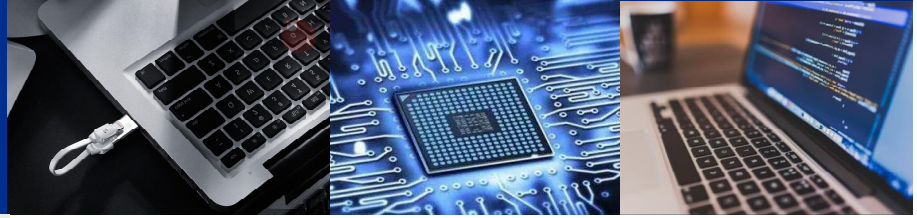
字扩展成双字指令CWD

格式：CWD ；

功能：将 AX 中的有符号数的符号位扩展到 DX 中，

DX，AX 中的数成为一个带符号双字

- **该类指令为隐含操作数**



3.3 8086指令系统

4. 十进制算术运算指令 （BCD 码调整指令）

调整指令的约定：

被调整的 BCD 码运算结果应放在 AL 中，调整后的正确结果返回AL中

- **该类指令为隐含操作数**
- **有六种调整指令**

① 压缩BCD 码加法调整指令（不影响OF的状态）

格式：**DAA；**

指令功能：对在AL中两个压缩型BCD数相加的结果，调整成压缩型BCD数在AL中；

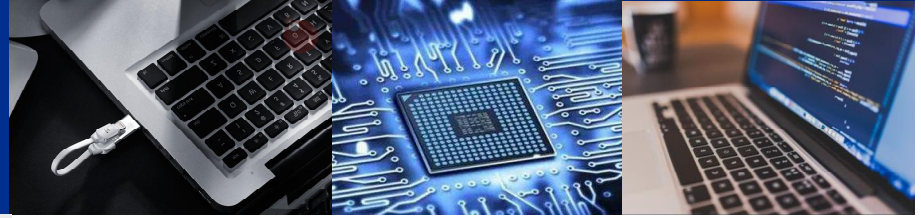
调整规律：AL低4位>9 或 AF=1　AL=AL+06H，AF=1

AL高4位>9 或 CF=1　AL=AL+60H，CF=1

注意点：必须紧跟在加法指令之后且只能对AL中的内容进行调整。

标志寄存器：影响CF、PF、AF、ZF、SF，其中CF=1说明结果大于99。

例：MOV BL，35H
MOV AL，85H
ADD AL，BL
DAA
结果 AL=20H，CF=1，
AF=1，PF=0，
ZF=0，SF=0



3.3 8086指令系统

② 压缩 BCD 码减法调整指令（**不影响OF的状态**）

格式：

DAS：

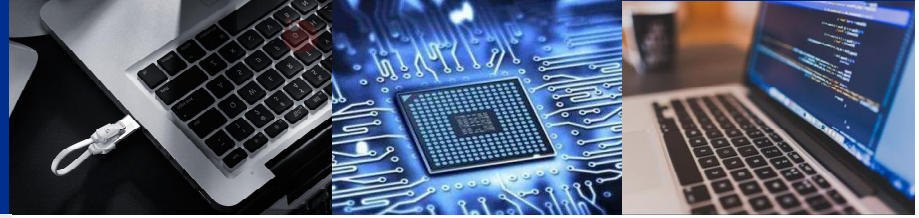
指令功能：对在AL中两个压缩型BCD数相减的结果，调整成压缩型BCD数在AL中；

调整规律：AL低4位>9 或 AF=1　AL=AL-06H，AF=1
　　AL高4位>9 或 CF=1　AL=AL-60H，CF=1

标志寄存器：同DAA指令。

注意点：必须紧跟在减法指令之后且只能对AL中的内容进行调整。

```
例：MOV     AL，63H          ；（AL）=63H，表示压缩BCD码63
    MOV     BL，28H          ；（BL）=28H，表示压缩BCD码28
    SUB     AL，BL           ；二进制减法：（AL）=63H - 28H=3BH
    DAS                      ；十进制调整：（AL）=35H，实现压缩
                BCD码减法：63 - 28=35
```



3.3 8086指令系统

非压缩BCD码调整指令

① 非压缩BCD码加法调整指令（只影响AF和CF的状态）

格式：AAA；

指令功能：对在AL中两个非压缩型BCD数相加的结果，调整成非压缩型BCD数在AL中；

若AL低4位大于9或AF=1，则AL+6，AH+1，CF和AF置1，AL高4位清0；

否则AL高4位清0，CF和AF置0，

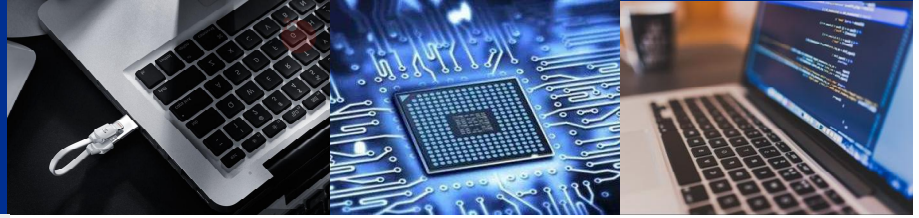
标志寄存器：该指令不影响PF、ZF、SF、OF。

注意点：

✓必须紧跟在ADD，ADC，INC指令之后且只能对AL中内容进行调整。

✓调整后 AL 中高四位为零， 如果 AF = 1 则使 CF =1 ， AH+1 → AH

```
例：MOV AX，0005H
    MOV BL，08H
    ADD AL，BL
    AAA
    结果AL=03H，AH=AH+1，CF=1，AF=1
```



3.3 8086指令系统

②非压缩BCD码减法调整指令（只影响 AF 和 CF ）

格式：AAS；

- 调整后 AL 中高四位清零，如果 AF = 1 ，CF = 1，AH-1 → AH
- 此指令应紧跟在 SUB，SBB，DEC 指令之后

例：SUB AL，BL ；AL和BL中的非压缩BCD数相减 （05H-07H）
AAS ；调整AL为正确的非压缩BCD数 （AL=08H，AH=AH-1）

③非压缩BCD 码乘法调整指令（影响 SF，ZF，PF）

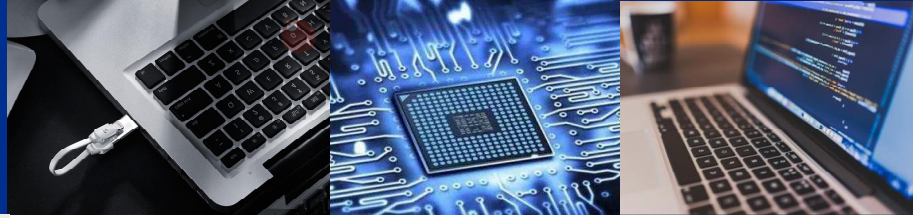
格式：AAM；

- 调整的结果在 AX 中，AH 和 AL 中分别为 高位 和 低位 非压缩BCD码
- 此指令应紧跟在 MUL 指令之后（AL/10 商→AH,余数→ AL）

④非压缩 BCD 码除法调整指令（影响 SF，ZF，PF）

格式：AAD；

- AX 中为两个非压缩的 BCD 码（被除数）
- 调整指令 AAD 应放在 除法指令 DIV 之前--- AH × 10+AL→AL
- 商在 AL 中，余数在 AH 中。



3.3 8086指令系统

4.3.3逻辑运算指令

操作数为字或字节，运算是按二进制位进行

① 逻辑非： NOT dst ; $\overline{dst} \rightarrow dst$

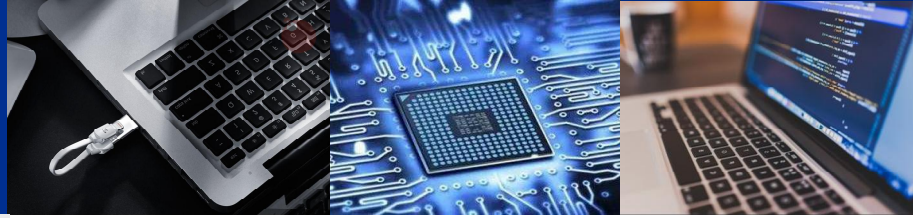
② 逻辑与： AND dst，src ； $dst \wedge src \rightarrow dst$

③ 逻辑或： OR dst，src ； $dst \vee src \rightarrow dst$

④ 逻辑异或： XOR dst，src ； $dst \oplus src \rightarrow dst$

⑤ 测试指令：TEST dst，src ； $dst \wedge src$ 只影响标志

- NOT 指令不影响 FR
- 其它指令影响 FR，但 AF 不定



3.3 8086指令系统

【例】

1．逻辑非

```
MOV     AX，878AH       ；  (AX) =878AH
NOT     AX，            ；  (AX) =7875H
```

2．将AL中第3位和第7位清零

```
MOV     AL，0FFH
AND     AL，77H
```

3．将AL寄存器中第3位和第7位置1

```
MOV     AL，0
OR      AL，88H
```

4．按位加运算

```
MOV     AL，45H         ；  (AL) =45H
XOR     AL，31H         ；  (AL) =74H
```

5．测试AX中的第12位是否为0，不为0则转L

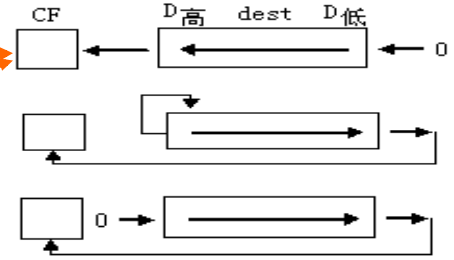
```
TEST      AX，1000H
JNE       L
```

4.3.4移位指令

dst ={ REG，MEM } 可以是字，也可以是字节

Shift arithmetic left

（1）算术左移：SAL dst, CNT；
（2）逻辑左移：SHL dst, CNT；
（3）算术右移：SAR dst, CNT；
（4）逻辑右移：SHR dst, CNT；

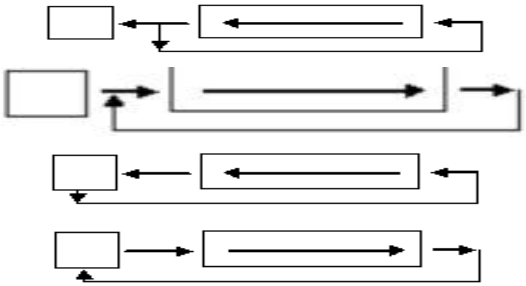


移动一位后，若(
另外，指令还影响

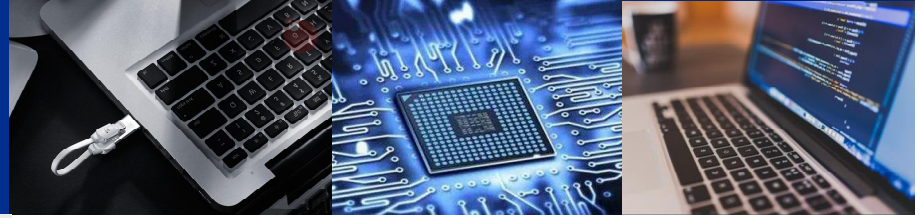
Shift logic left

rotate left

（1）循环左移：ROL dst, CNT；
（2）循环右移：ROR dst, CNT；
（3）带CF循环左移：RCL dst, CNT；
（4）带CF循环右移：RCR dst, CNT；



- 移一位时，CNT = 1；当移动多位时，CNT 应为 CL，CL中的数为移位位数



4.3.5串操作指令

串操作指令就是用一条指令实现对一串字符或数据的操作。

使用该类指令时，有如下约定：

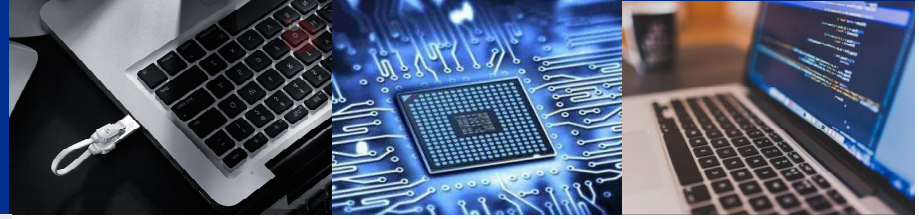
- 源串在数据段中：用 SI 指示串首元素在数据段的有效地址。
- 目的串在扩展段中：用 DI 指示目的串首元素在扩展段内的有效地址。
- 每做一次串操作：自动修改指针SI 和 DI ，使指针指向下一个串元素。

DF = 0　指针值增加
用CLD指令使 DF=0

DF = 1　指针值减小
用STD指令使 DF=1

字节串操作时	字串操作时
SI+1 → SI	SI+2 → SI
DI+1 → DI	DI+2 → DI
SI–1 → SI	SI–2 → SI
DI–1 → DI	DI–2 → DI

- 隐含使用了[SI]，[DI]，或AX，AL作为 dst 或 src



3.3 8086指令系统

1.基本串操作指令

(1) 串传送指令： ● **不影响FR** ● **src：MEM； dst：MEM**

① 字节串传送指令：

格式： MOVSB； [DS×16＋SI] B → [ES×16＋DI]

● 自动修改指针： SI±1 → SI DI±1 → DI

② 字串传送指令：

格式： MOVSW； [DS×16＋SI] W → ES×16＋DI]

• 自动修改指针： SI±2 → SI DI±2 → DI

如何实现同一段内传输？

(2) 串比较指令： • **影响FR**

① 字节串比较指令：

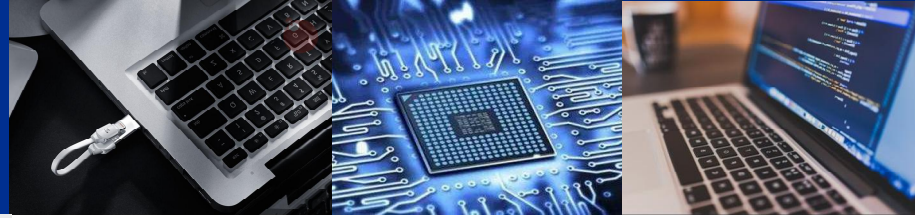
格式： CMPSB； [DS×16＋SI] － [ES×16＋DI]

• 自动修改指针： SI±1 → SI DI±1 → DI

② 字串比较指令：

格式： CMPSW； [DS×16＋SI] － [ES×16＋DI]

• 自动修改指针： SI±2 → SI DI±2 → DI



3.3 8086指令系统

(3) 串扫描指令：　● 影响 FR　● src：AL/AX；　dst：MEM

① 字节串扫描：
格式：SCASB；　AL - [ES×16 + DI] 且 DI ± 1 → DI

② 字串扫描：
格式：SCASW；　AX - [ES×16 + DI] 且 DI ±2 → DI

(4) 串装入指令：　● 不影响 FR　● src：MEM；　dst：AL/AX

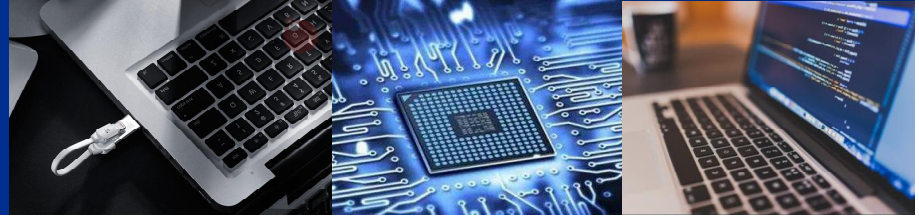
① 字节装入
格式：LODSB；　[DS×16 + SI] → AL 且 SI±1 → SI

② 字串装入：
格式：LODSW；　[DS×16 + SI] → AX 且 DI±2 → DI

(5) 串贮存指令：　● 不影响 FR　● src：AL/AX；　dst：MEM

① 字节存储：
格式：STOSB；　AL → [ES×16 + DI] 且 DI ± 1 → DI

② 字串存储：
格式：STOSW；　AX → [ES×16 + DI] 且 DI ±2 → DI



把当前数据段中偏移1000H开始的100个字节的数据传送到从偏移2000H开始的单元中

```
CLD
    PUSH   DS
    POP    ES
    MOV    SI,1000H
    MOV    DI,2000H
    MOV    CX,100
NEXT: LODSB
    STOSB
    DEC    CX
    JNZ    NEXT
```

(1) lodsb、lodsw：把DS:SI指向的存储单元中的数据装入AL或AX，然后根据DF标志增减SI

(2) stosb、stosw：把AL或AX中的数据装入ES:DI指向的存储单元，然后根据DF标志增减DI

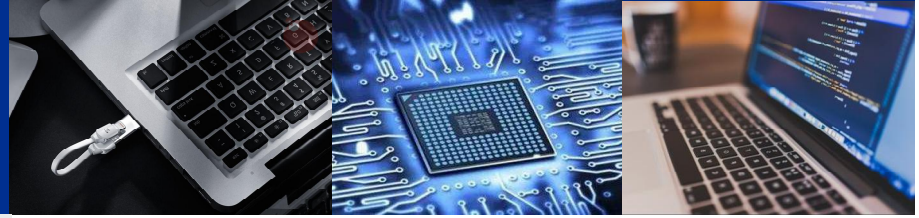
(3) movsb、movsw：把DS:SI指向的存储单元中的数据装入ES:DI指向的存储单元中，然后根据DF标志分别增减SI和DI

(4) scasb、scasw：把AL或AX中的数据与ES:DI指向的存储单元中的数据相减，影响标志位，然后根据DF标志分别增减SI和DI

(5) cmpsb、cmpsw：把DS:SI指向的存储单元中的数据与ES:DI指向的存储单元中的数据相减，影响标志位，然后根据DF标志分别增减SI和DI

(6) rep：重复其后的串操作指令。重复前先判断CX是否为0，为0就结束重复，否则CX减1，重复其后的串操作指令。主要用在MOVS和STOS前。一般不用在LODS前。

上述指令涉及的寄存器：段寄存器DS和ES、变址寄存器SI和DI、累加器AX、计数器CX　　涉及的标志位：DF、AF、CF、OF、PF、SF、ZF



3.3 8086指令系统

2. 串操作 重复前缀 指令

- 重复前缀用来控制紧跟在其后的字符串操作指令是否重复执行
- CX 寄存器中的值为重复的次数

(1) 无条件重复前缀：

- **用在MOVS，STOS，LODS指令前**

格式：REP

功能：对跟在REP后面的串操作指令重复执行

- CX ≠ 0 时，重复执行串操作。每执行一次串操作，CX - 1 → CX，当 CX = 0时，停止执行串操作，转向执行下一条指令
- 用于存储器中数据块的搬家

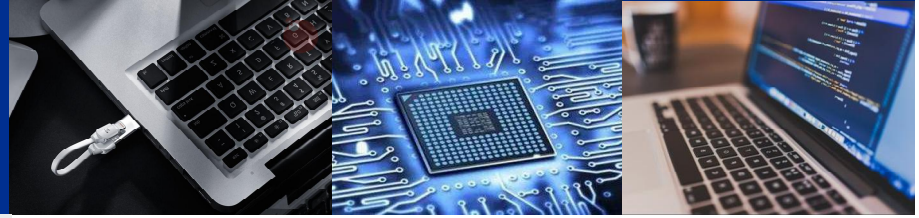
(2) 相等重复前缀：

- **用在CMPS，SCAS指令前**

格式：REPE

功能：对跟在REPE后面的串操作指令重复执行

- 条件为：CX ≠ 0 且 ZF = 1。
- 每执行一次串的比较或扫描的操作，CX - 1 → CX，当满足上述条件，重复执行串操作 。当 ZF = 0 或 CX = 0 时，停止串操作的执行 。



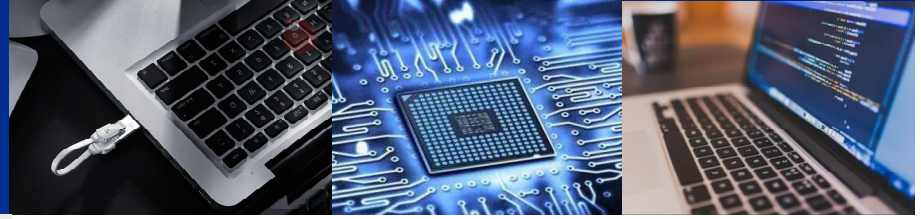
3.3 8086指令系统

③ 不相等重复前缀： • **用在CMPS，SCAS指令前**

格式：REPNE （或REPNZ）

功能：对跟在REPNZ后的串操作指令重复执行

- **条件为：CX ≠ 0 且 ZF = 0。**
- 每执行一次串的比较或扫描的操作，CX - 1 → CX，当满足上述条件，重复执行串操作。直至 CX = 0 或 ZF = 1 时，停止串操作的执行。
- 用于比较两个串中的元素都**不相等**或在**串中扫描某个元素**



3.3 8086指令系统

4.3.6程序控制指令

- 8086 的指令系统共有五类转移指令:无条件转移； 条件转移； 循环控制； 过程调用和返回； 中断

(1) 无条件转移指令JMP

基本格式：JMP target；　　target：指令中指出的转移目标地址操作数

功能：使程序的执行无条件转移到该指令指出的目标地址target去取指令并执行。

- 根据转移距离（距该转移指令）的远近，JMP 指令又分成下面几种类型：

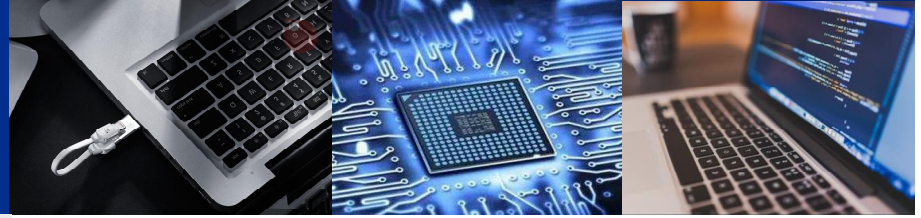
① **段内直接短转移：（相对转移）**

格式：JMP　short target;　　short: 属性操作符，“短”

功能：把当前的 IP 值与 target 代表的8位偏移量之和送入（IP），即：（IP + target）→ IP；

CPU从 IP 新指出的地址开始取指令并执行，程序的运行发生转移。

- target：8 位带符号数，因此转移的范围以本指令为中心在：-128—+ 127字节内。通常用标号（标号地址）代替。
- 标号地址：（符号地址）以字母起头，可代替8位带符号的偏移量。



3.3 8086指令系统

② 段内直接近转移：（相对转移）

格式：JMP NEAR PTR target;

功能：（IP + target）⟶IP；使程序的运行发生转移，以本指令为中心在：
- 32768 ～ + 32767字节范围内转移,可转到段内任一地址。

- NEAR：标号地址的距离属性，“近”；
- PTR：属性操作符；

距离属性 NEAR 通常可以省略，如： JMP target；

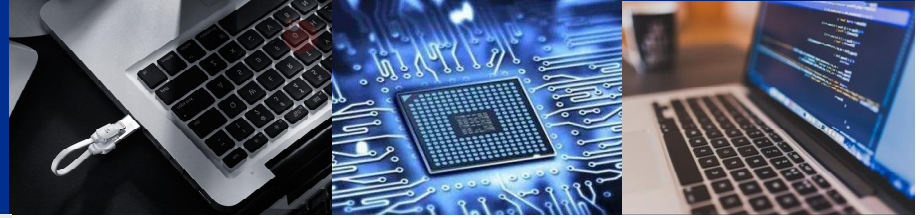
- target：标号地址 = disp16

③ 段内间接转移

格式：JMP REG / MEM ; 通过REG/MEM间接给出转移的目标地址。

功能：（REG / MEM）⟶IP；即将寄存器中的值或存贮器中的一个字送到 IP 中，使程序的运行发生转移。

例：
```
JMP  BX;
JMP  [ BX+25H];
JMP  [ 0125H]
```



3.3 8086指令系统

④ 段间直接转移：

格式：JMP FAR PTR target；　　target：符号地址或32位的立即数构成

功能：将符号地址的前面一个字送 IP，后一个字送 CS；这样同时修改了CS和 IP中的值，程序的执行跳转到另一代码段开始执行。

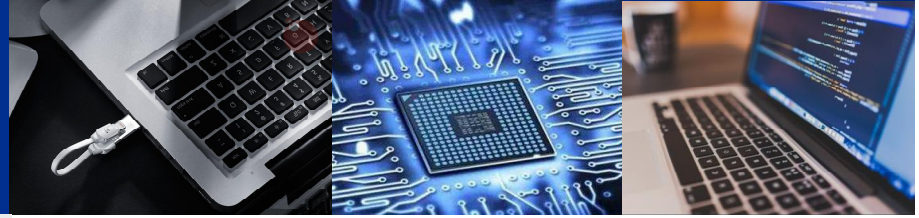
- FAR：地址的距离属性 “ 远 ”
- 例：　JMP　21000100H

⑤ 段间间接转移

格式：JMP DWORD PTR［MEM］；　　[MEM]：操作数的寻址方式

功能：由指令中的存储器操作数寻址方式提供 MEM 的有效地址EA，此EA指定的双字中，第一个字送 IP，第二个字送 CS，从而得到转移的目标地址；CPU从另一代码段取指并开始执行。

- DWORD：变量类型，“ 双字 ”
- 例：　JMP　DWORD　PTR [2100H]
- 例：　JMP　DWORD　PTR [BX]



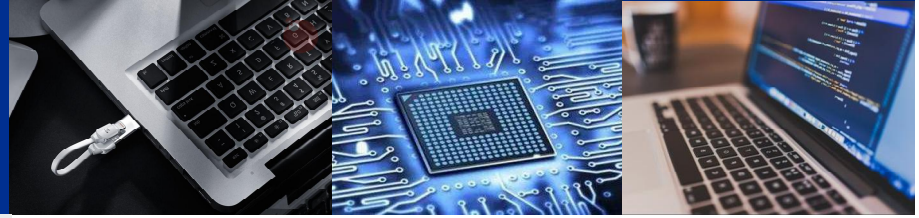
(2) 条件转移指令

- 该类指令属段内直接短转移；指令中的标号地址为 8 位带符号的偏移量。

- 该类指令对状态标志位进行测试，当满足要求的条件时，则发生转移；
  即： (IP + target) ⟶ IP。否则， 顺序执行下一条指令。

- 条件转移指令共有四种类型：

I. **单个标志位**的条件转移；
II. 两个**无符号数比较**的条件转移；
III. 两个**有符号数比较**的条件转移；
IV. 测试 **CX = 0** 的条件转移；



3.3 8086指令系统

I. 单个标志位的条件转移指令：

① **测试ZF**：
JZ / JE target ; ZF= 1 结果为零(两数比较相等)，则转移
JNZ / JNE target ; ZF= 0 结果不为零(比较不相等)，则转移

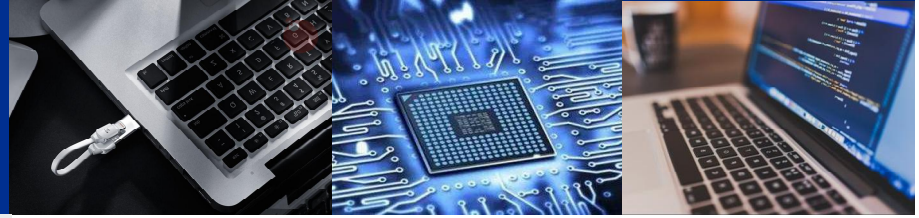
② **测试CF**：
JC target ; CF= 1 结果有进位/借位，则转移
JNC target ; CF= 0 结果无进位/借位，则转移

③ **测试SF**：
JS target ; SF= 1 结果为负时，则转移
JNS target ; SF= 0 结果为正时，则转移

④ **测试OF**：
JO target ; OF= 1 结果有溢出，则转移
JNO target ; OF= 0 结果无溢出，则转移

⑤ **测试PF**：
JP target ; PF= 1 结果中1的个数为偶数，则转移
JNP target ; PF= 0 结果中1的个数为奇数，则转移

- 当条件成立时，发生转移；即： (IP + target) → IP
- target 标号地址为 8 位带符号的偏移量。（后同）



3.3 8086指令系统

II. 两个无符号数比较转移指令：一般跟在CMP指令后

①格式： JA / JNBE target ；dst > src 则转移；
②格式： JAE / JNB target ；dst ≥ src则转移；
③格式： JB / JNAE target ；dst < src则转移；
④格式： JBE / JNA target ；dst ≤ src则转移；

- 当条件成立时，发生转移；即： (IP + target) → IP

III. 两个有符号数比较转移指令：也应紧跟在CMP指令后

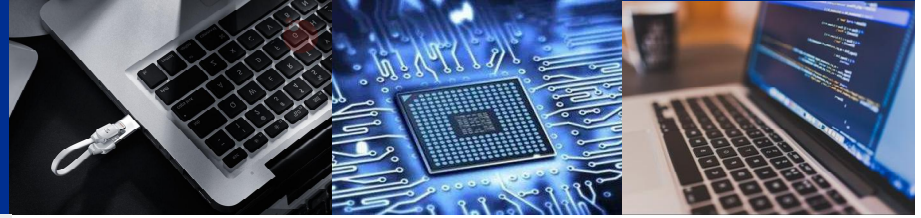
①格式： JG / JNLE target ；dst > src 则转移；
②格式： JGE / JNL target ；dst ≥ src则转移；
③格式： JL / JNGE target ；dst < RC 则转移；
④格式： JLE / JNG target ；dst ≤ src则转移

- 当条件成立时，发生转移；即： (IP + target) → IP

IV. 测试CX：

格式： JCXZ target ；CX=0，则转移

- 当条件成立时，发生转移；即： (IP + target) → IP



3.3 8086指令系统

(3) 循环控制指令

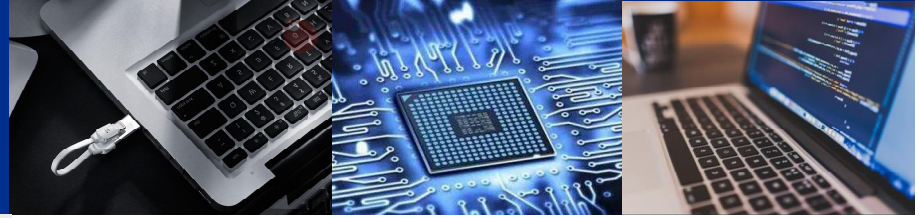
- 当需要循环执行某段程序时（此段程序称为循环体），用此类指令， 可根据设置的循环条件，决定是否循环执行循环体。
- 该类指令一般放在循环体的尾部或首部，执行该指令时对设置的循环条件判断；
  当满足条件，则转移到该指令中指明的标号地址开始执行循环体；
  不满足条件，顺序执行。

- 该类指令属段内直接短转移；指令中的标号地址为 8 位带符号的偏移量。

① 计数循环指令：

格式：LOOP　disp8　　；CX ≠ 0，（IP + disp8）→ IP

功能：CX 为循环次数计数器，循环次数置入CX中，
每执行一次 LOOP 指令，CX - 1 → CX，
如果 CX ≠ 0，则 IP + disp8 → IP，即发生转移；
如果 CX = 0，顺序执行。



3.3 8086指令系统

② 结果为零或相等时循环指令：　一般用在CMP指令后

格式：LOOPZ　disp8　；CX≠0 且 ZF = 1时，　(IP + disp8) → IP
　　LOOPE　disp8

功能：每执行一次该指令，CX - 1→CX，判 CX≠0 且 ZF = 1时，
　　则产生转移，即：　(IP + disp8) → IP

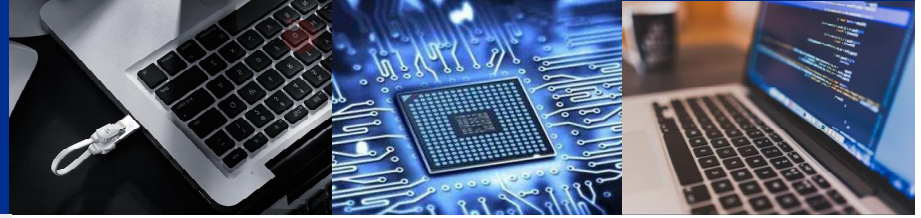
- ZF = 1，说明前一条比较指令比较的结果相等，即结果为零。

③ 结果不为零或不相等时循环指令：一般用在CMP指令

格式：LOOPNZ　disp8；CX≠0 且 ZF = 0时，　(IP + disp8) → IP
　　LOOPNE　disp8

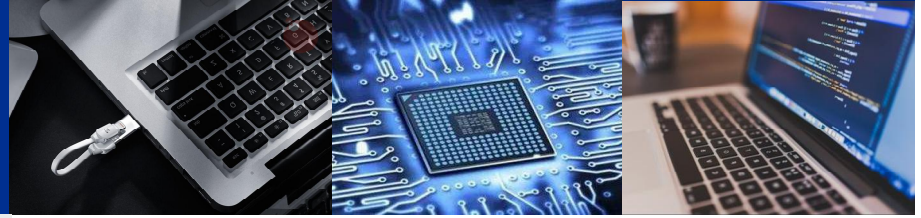
功能：每执行一次该指令，CX - 1→CX，判 CX≠0 且 ZF = 0 时，
　　则产生转移，即：　(IP + disp8) → IP

- ZF = 0，说明前一条比较指令比较的结果不相等。
- 以上三类指令均以CX寄存器为计数器



2. 子程序调用和返回

- 子程序：将某些经常使用的，并具有一定功能的程序段，单独组成一个 程序（模块），**独立**的放在代码段中某一处；这个模块称为：**“子程序” 或“过程”**。
  - **子程序名：子程序的 标号地址。（也称过程名）**
  - 使用子程序：要用专用指令来调用，即让程序运行跳转到子程序运行。
  - 子程序执行完毕：要返回到调用指令的下一条指令处去顺序执行，即：子程序返回；因此在子程序的结尾处，要安排子程序返回指令RET\RETF。
  - RET 指令其核心也是一种转移。
  - 调用指令基本格式： CALL 子程序名
  - 根据子程序所处的位置距调用指令CALL 的 远近，调用有四种格式



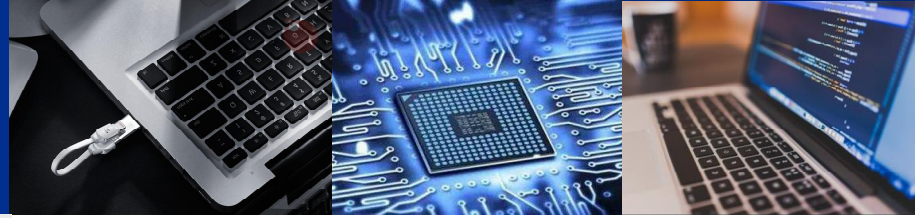
3.3 8086指令系统

(1) 段内直接调用：（相对调用）

格式：CALL　disp16；16位相对偏移地址disp16
功能：① IP → 堆栈；即将IP的内容压栈，保存返回地址，即断点。
其内容为CALL指令下一条指令的有效地址EA。
② IP + disp16 → IP；将指令中的disp16和IP 相加后送 IP，
使程序的执行发生转移。
③ disp16可以使用标号地址。（过程名）

(2) 段内间接调用：（绝对调用）

格式：CALL REG / MEM
功能：① IP → 堆栈；即将IP 的内容压栈，保存返回地址，即断点。
② (EA) → IP； EA 是由REG / MEM操作数指出的子程序在
本代码段内的有效地址。从而使程序的运行发生转移。
③ EA 由寄存器或存贮器提供。



3.3 8086指令系统

(3) 段间直接调用：（子程序不在当前代码段）

格式：CALL　addr32　　;(addr32：或远过程名)

功能：① CS →堆栈；IP →堆栈；保存返回地址，即断点。

② addr32：32位地址中第一个字 →IP；第二个字 →CS；

程序的运行转移到另一个代码段中，开始执行子程序。

(4) 段间间接调用：（子程序不在当前代码段）

格式：CALL　MEM　;

功能：① CS → 堆栈；IP → 堆栈；保存返回地址，即断点。

② 指令提供了四个字节（双字）MEM 操作数的寻址方式

③ 存贮器中的第一个字 → IP；第二个字 → CS；程序运行发生转移。

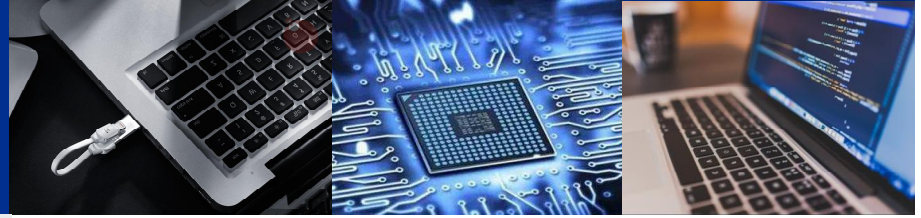
(5) 子程序返回指令：（该指令写在子程序最后）

格式：RET/RETF　　；将栈顶原保护的断点出栈

功能：① 段内返回：（SP）→ IP；SP + 2 →SP：修改栈指针

② 段间返回：（SP）→ IP；SP + 2 → SP：修改栈指针

（SP）→ CS；SP + 2 →SP：修改栈指针



3.3 8086指令系统

3. 中断和中断返回指令

(1) 软中断指令：

格式：INT　n　；n 为中断类型号，八位二进制数0 - 255。

功能：① 置 IF = 0；关闭中断，不允许外部中断的干扰；

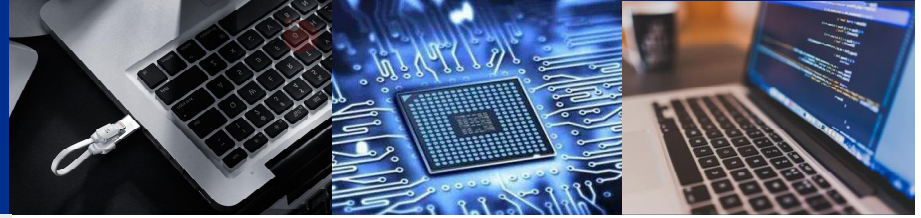
② FR → 栈区；保护现场，即中断服务前FR的状态；

③ CS → 栈区； IP → 栈区；保护断点：即返回地址；

④ n → CS； [CS×16 + n×4] → IP；修改IP指针

[CS×16 + n×4 + 2] → CS；修改段基地址

⑤ CPU 转去执行相应的中断服务服务程序。



3.3 8086指令系统

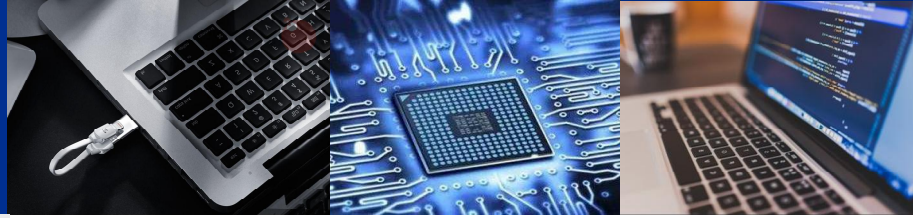
(2) 中断返回指令：（该指令写在中断服务程序结尾处）

格式：IRET　　；其作用类似子程序返回指令

功能：① (SP) → IP，SP + 2 → SP；弹出断点：即返回地址
(SP) → CS，SP + 2 → SP；
(SP) → FR，SP + 2 → SP；恢复现场

② 置 IF = 1；开放中断允许。

③ 由于出栈修改了 IP 和 CS，即恢复断点的地址，使CPU返回原断点处，继续执行主程序。



3.3 8086指令系统

3.3.7处理器控制指令

该类指令用来控制8086CPU的状态标志、控制标志以及使CPU与 外部设置同步，协调CPU与协处理器之间的同步。

1. 标志位设置指令：

① 格式：CLC ；功能：使 CF = 0，即进位标志清零。

② 格式：STC ；功能：使 CF = 1，即进位标志置1。

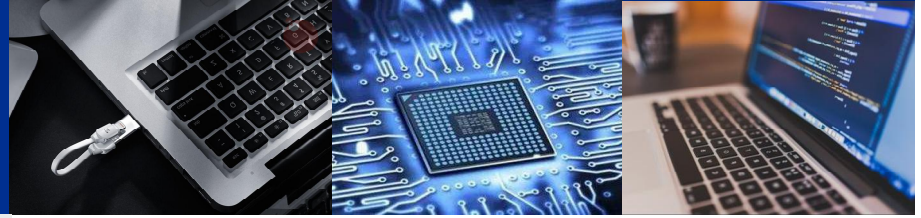
③ 格式：CMC ；功能：使 $\overline{CF} \rightarrow CF$，即进位标志取反。

④ 格式：CLD ；功能：使 DF = 0；即方向标志位清零。

⑤ 格式：STD ；功能：使 DF = 1；即方向标志位置1。

⑥ 格式：CLI ；功能：使 IF = 0；中断标志清零（关中断）

⑦ 格式：STI ；功能：使 IF = 1；中断标志置1（开中断）



3.3 8086指令系统

2. 同步指令

在最大模式系统时，CPU用同步指令向协处理器发出请求，协调工作

① 处理器交权指令ESC

格式：ESC　ext-op , mem

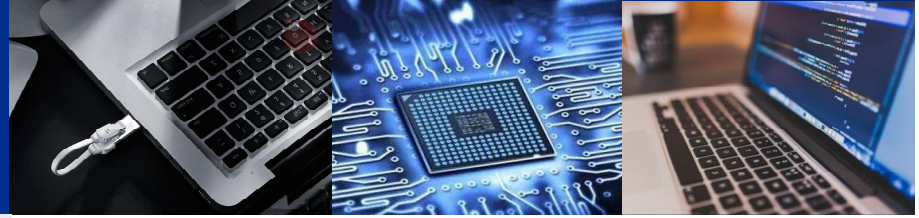
功能：使外部处理器能从8086指令流中取得它们的操作指令，同时指示8086CPU取出内存操作数，放到数据总线上，一供使用。

- ext-op：其它处理器的操作码，称外操作码；　mem：内存操作数

② 等待指令WAIT

格式：WAIT

功能：该指令与ESC指令配合使用，在WAIT指令期间，CPU测试TEST 引脚状态；当TEST有效时，退出等待状态执行后续指令，否则继续等待。 TEST 信号由协处理器发出。



3.3 8086指令系统

③ **总线封锁前缀指令LOCK**：是一个前缀，应放在其它操作指令前

格式：LOCK；

功能：封锁总线。在执行指令期间，不允许其它设备对总线进行访问

④ **暂时指令HLT**

格式：HLT

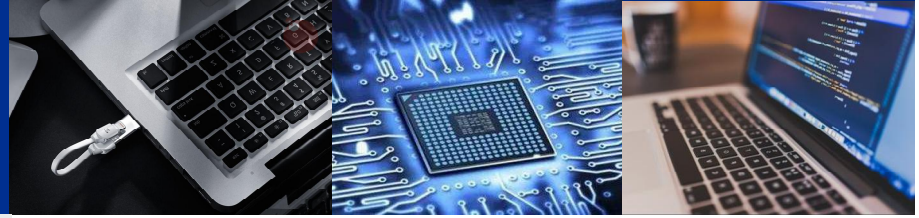
功能： CPU处理暂停状态，等待外设的中断请求。

- 由RESTET，NMI，INTR信号可使CPU退出暂停状态

⑤ **空操作指令NOP**

格式：NOP

功能：不产生任何结果，用于延时3个T周期



3.4 例题解析

1. 指出下列指令的错误。

(1)MOV AH,BX

(2)MOV [SI] , [BX]

(3) MOV AX, [SI] [DI]

(4) MOV AX, [BX] [BP]

(5) MOV [BX] , ES: AX

(6) MOV BYTE PTR [BX] ,1000

(7) MOV AX,OFFSET [SI]

(8) MOV CS,AX

(1) 源、目的操作数字长不一致。

(2) 源、目的操作数不能同时为存储器寻址方式。

(3) 基址变址方式不能有 SI和DI的组合。

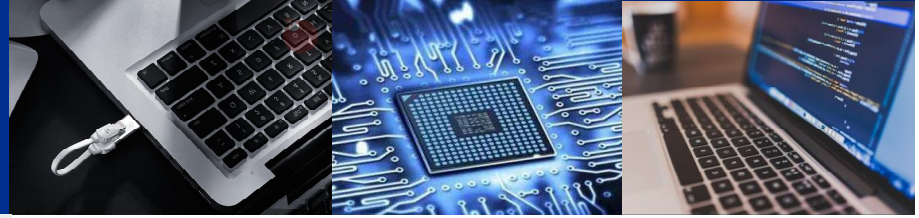
(4) 基址变址方式不能有 BX和BP的组合。

(5) 在8086寻址方式中，AX不能作为基址寄存器使用,而且源、目的操作数不能同时为存储器寻址方式。

(6) 1000超出一个字节的表数范围。

(7) OFFSET只用于简单变量，应去掉。

(8) CS不能作为目的寄存器。



3.4 例题解析

2. 假设(CS)=3000H,(DS)=4000H,(ES)=2000H,(SS)=5000H,(AX)=2060H,(BX)=3000H,(CX)=5,(DX)=0,(SI)=2060H,(DI)=3000H,(43000H)=0A006H,(23000H)=0B116H,(33000H)=0F802H,(25060)=00B0H,(SP)=0FFFEH,(CF)=1,(DF)=1,请写出下列各条指令单独执行完后,有关寄存器及存储单元的内容,若影响条件码，请给出条件码SF、ZF、OF、CF的值。

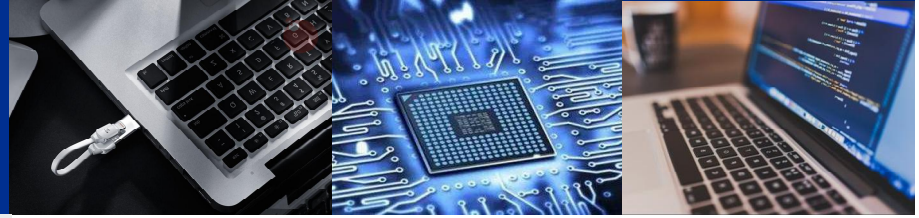
(1)SBB　AX,BX　　(1) (AX)=0F05FH,(SF)=1,(ZF)=0,(OF)=0,(CF)=1。

(2)CMP　AX,WORD PTR［SI+0FA0H］　　(2) (SF)=1,(ZF)=0,(OF)=1,(CF)=1。

(3) MUL　BYTE PTR［BX］　　(3) (AX)=0240H,(OF)=1,(CF)=1。

(4) AAM　　(4) (AX)=0906H,(SF)=0,(ZF)=0。

(5) DIV　BH　　(5) (AX)=20ACH



3.4 例题解析

2. 假设(CS)=3000H,(DS)=4000H,(ES)=2000H,(SS)=5000H,(AX)=2060H,(BX)=3000H,(CX)=5,(DX)=0,(SI)=2060H,(DI)=3000H,(43000H)=0A006H,(23000H)=0B116H,(33000H)=0F802H,(25060)=00B0H,(SP)=0FFFEH,(CF)=1,(DF)=1,请写出下列各条指令单独执行完后,有关寄存器及存储单元的内容,若影响条件码，请给出条件码SF、ZF、OF、CF的值。

(6) SAR AX,CL

(7) XOR AX,0FFE7H

(8) REP STOSB

(9) JMP WORD PYR [BX]

(10) XCHG AX,ES: [BX+SI]

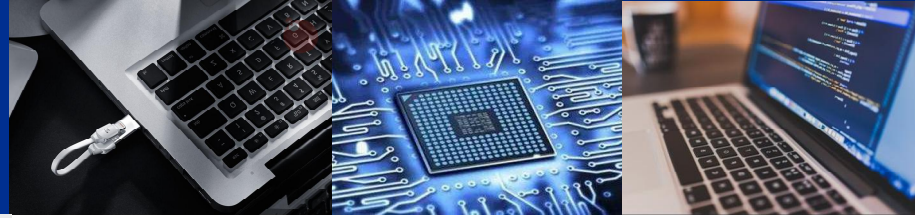
(6) (AX)=0103H,(CF)=0。

(7) (AX)=0DF87H,(CF)=0,(OF)=0,(SF)=1,(ZF)=0。

(8) (23000H)~(23004H)=60H,不影响标志位。

(9) (IP)=0A006H,不影响标志位。

(10) (AX)=00B0H,(25060)=2060H,不影响标志位。



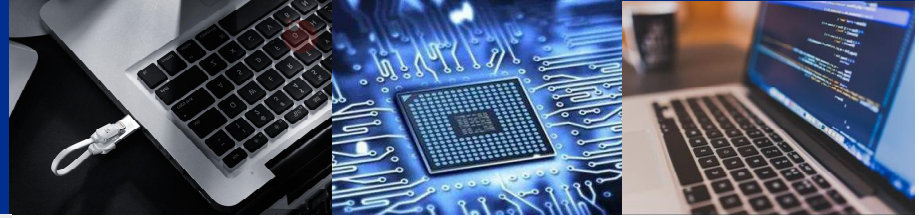
3.4 例题解析

3. 试分析下面的程序段完成什么操作。

```
MOV     CL,04
SHL     DX,CL
MOV     BL,AH
SHL     AX,CL
SHR     BL,CL
OR      DL,BL
```

【解析】

该程序段是在做一道乘法：(DX:AX)×16，即将双字DX:AX左移4位；乘积高位送入DX，低位存入AX，不计高位DX×16的溢出值。



3.4 例题解析

4. 用其他指令完成和下列指令一样的功能。

(1)REP MOVSB

(2)REP LODSB

(3)REP STOSB

(4)REP SCASB

【解析】

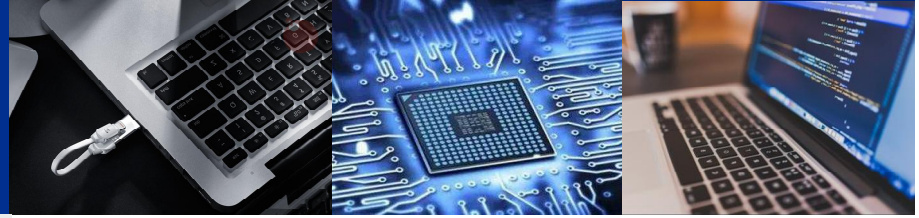
MOVSB把DS:SI指向的存储单元中的一个字节装入ES:DI指向的存储单元中，然后根据DF标志分别增减SI和DI；

LODSB把DS:SI指向的存储单元中的数据装入AL，然后根据DF标志增减SI；

STOSB把AL中的数据装入ES:DI指向的存储单元，然后根据DF标志增减DI；

SCASB把AL中的数据与ES:DI指向的存储单元中的数据相减，影响标志位，然后根据DF标志分别增减SI和DI

REP重复其后的串操作指令。重复前先判断CX是否为0，为0就结束重复，否则CX减1，重复其后的串操作指令。



3.4 例题解析

(1) REP MOVSB

```
LOOP1:
        MOV   AL,BYTE PTR  [SI]
        MOV   ES: BYTE PTR  [DI] , AL
        INC     SI  (或DEC  SI)
        INC     DI  (或DEC  DI)
        LOOP   LOOP1
```

(2)REP LODSB

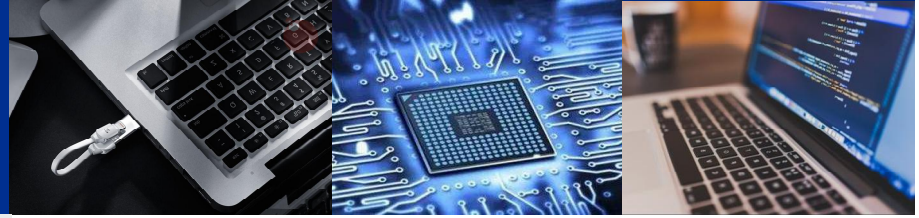
```
LOOP1:
        MOV   AL,BYTE PTR ［SI］
        INC     SI (或DEC  SI)
        LOOP  LOOP1
```

(3) REP STOSB

```
LOOP1:
        MOV ES: BYTE PTR [DI］,AL
        INC DI (或DEC DI)
        LOOP LOOP1
```

(4) REP SCASB

```
LOOP1:
        CMP   AL,ES: BYTE PTR ［DI］
        JE      EXIT
        INC    DI (或DEC   DI)
        LOOP  LOOP1
EXIT:
```



3.4 例题解析

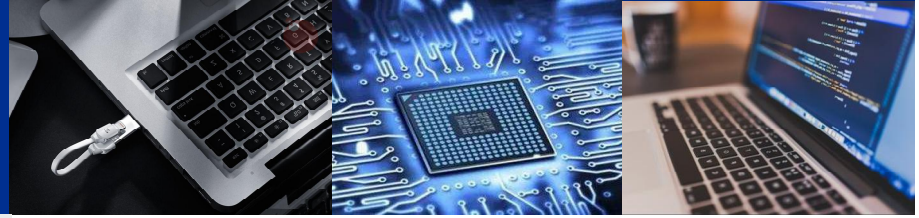
5. 编写程序段,比较两个5字节的字符串OLDS和NEWS,如果OLDS字符串与NEWS不同,则执行NEW_LESS,否则顺序执行程序。

【解析】

首先取得源字符串的首地址送SI，目标字符串的首地址送DI，给CX送串的长度5。假设地址变化的方向是增址变化，利用CMPSB进行串的比较，相等重复串比较，不相等则转移到NEW_LESS。

程序段如下：

```
LEA     SI,OLDS
LEA     DI,NEWS
MOV     CX,5
CLD
REPZ    CMPSB
JNZ     NEW_LESS
```



3.4 例题解析

6. 假定AX和BX中的内容为带符号数,CX和DX中的内容为无符号数,请用比较指令和条件转移指令实现以下判断:

(1) 若DX的值超过CX的值,则转去执行EXCEED。

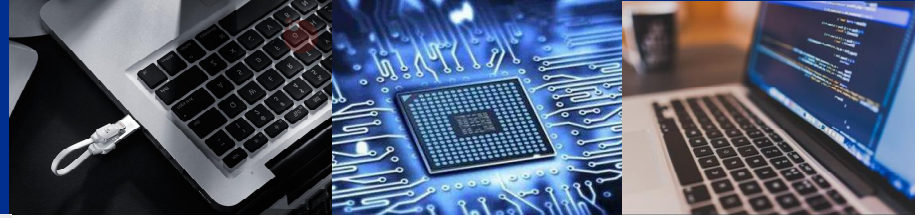
```
CMP     DX,CX
JA      EXCEED
```

(2) 若BX的值大于AX的值,则转去执行EXCEED。

```
CMP     BX,AX
JG      EXCEED
```

(3) CX中的值为0吗? 若是则转去执行ZERO。

```
CMP     CX,0
JE      ZERO
```



3.4 例题解析

(4) BX的值与AX的值相减,会产生溢出吗? 若溢出则转OVERFLOW。

```
SUB     BX,AX
JO      OVERFLOW
```

(5) 若BX的值小于AX的值,则转去执行EQ_SMA。

```
CMP     BX,AX
JL      EQ_SMA
```

(6) 若DX的值低于CX的值,则转去执行EQ_SMA。

```
CMP     DX,CX
JB      EQ_SMA
```